

Lab Report

BCSE307P- Compiler Design Laboratory

Slots: L29+L30

CH2025260502087

School of Computer Science and Engineering

by

S Vishwajith 23BCE1145



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

April, 2026



BCSE307P –COMPILER DESIGN LABORATORY

Slots: L29+L30 CH2025260502087 (Friday- 11.40 am to 1.20 pm)

Course faculty: Dr.R.Suganya, Professor, SCOPE, VIT

Venue: AB3-310 – D1 slot

Course Outcomes	Content	Page number	Date of submission
1	a) Recognition of Tokens	3	14.12.2025
	b)Regular Expressions	10	17.12.2025
	a)RE to DFA	22	24.12.2025
	b)RE to NFA	38	24.12.2025
2	a) Predictive Parsing	43	14.01.2026
	b) Operator Precedence	54	02.02.2026
3	LEX Programs	61	11.02.2026
4	a)SLR () Parser	84	20.02.2026
	bCLR() Parser	102	04.03.2026
5	a)Three Address Code	119	11.03.2026
	b) TAC for Control flow structure	134	25.03.2026
	c)Code Optimization	163	27.03.2026
6	Model exam	10.4.26	
7	FAT Lab	17.4.26	

Signature of Course Faculty
Dr. R. Suganya

Lab Assignment

Name	S Vishwajith
Registration Number	23BCE1145
Subject	Compiler Design
Assignment Number	1

Aim:

To write a C++ program LA.cpp that acts as a lexical analyser to read a C++ source file Factorial.cpp containing a factorial program. The program should identify tokens such as keywords, identifiers, operators, constants, and symbols. It should then print each token with its type and value.

Algorithm:

1. Open the source file Factorial.cpp in read mode.
2. Repeat until End of File (EOF) is reached:
 - a. Read each word (separated with a blank space) from the file.
 - b. If the word has a special symbol, split it into 3 (before, special symbol, and after) and print each part as a token.
 - c. Check if the word is a keyword, operator or a special symbol and print accordingly.
 - d. If it's none of them, then the word is an identifier and print it accordingly.
3. Close the file after reading.

Codes:

Factorial.cpp:

```
#include <iostream>
using namespace std;
int factorial(int n)
{
    if (n <= 1)
        return 1;
    return n * factorial(n - 1);
}
int main()
{
```

```

int n;
cin >> n;
cout << factorial(n) << endl;
return 0;
}

```

LA.cpp:

```

#include <fstream>
#include <iostream>
#include <string>
#include <sstream>
#include <algorithm>
#include <vector>
using namespace std;
vector<string> keywords = {"include", "using", "namespace", "int", "if", "return"};
vector<string> operators = {"<", ">", "=", "-", ">>", "<<"};
vector<string> special_symbols = {"(", ")", ";", "{", "}", "#"};
bool hasPunctuator(string word)
{
    for (const auto &spec : special_symbols)
    {
        if (word.find(spec) != string::npos)
        {
            return true;
        }
    }
    return false;
}
void printToken(string word)
{
    if (word.length() > 1 && hasPunctuator(word))
    {
        for (const auto &spec : special_symbols)
        {
            size_t pos;
            while ((pos = word.find(spec)) != string::npos)
            {
                string beforePunct = word.substr(0, pos);
                if (!beforePunct.empty())
                {
                    printToken(beforePunct);
                }
                cout << "Type: SPECIAL SYMBOL, Value: " << spec << endl;
            }
        }
    }
}

```

```

        word = word.substr(pos + spec.length());
    }
}
if (!word.empty())
{
    printToken(word);
}
return;
}
if (word.find("<") != string::npos && word.find(">") != string::npos && word.length() > 1)
{
    cout << "Type: OPERATOR, Value: <" << endl;
    word.erase(remove(word.begin(), word.end(), '<'), word.end());
    word.erase(remove(word.begin(), word.end(), '>'), word.end());
    printToken(word);
    cout << "Type: OPERATOR, Value: >" << endl;
    return;
}
auto itk = find(keywords.begin(), keywords.end(), word);
auto ito = find(operators.begin(), operators.end(), word);
auto itp = find(special_symbols.begin(), special_symbols.end(), word);
if (itk != keywords.end())
{
    cout << "Type: KEYWORD, Value: " << word << endl;
}
else if (ito != operators.end())
{
    cout << "Type: OPERATOR, Value: " << word << endl;
}
else if (itp != special_symbols.end())
{
    cout << "Type: PUNCTUATOR, Value: " << word << endl;
}
else
{
    cout << "Type: IDENTIFIER, Value: " << word << endl;
}
}
int main()
{
    ifstream inputFile("Factorial.cpp");
    if (!inputFile.is_open())
    {
        cerr << "Error: Could not open the file 'Factorial.cpp'" << endl;
    }
}

```

```
        return 1;
    }
    string word;
    while (inputFile >> word)
    {
        printToken(word);
    }
    inputFile.close();
    return 0;
}
```

Output:

```
PS C:\Users\vishw\Coding\Compiler-Lab> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-1\" ; if ($?) { g++ LA.cpp -o LA } ; if ($?) { .\LA }
Type: SPECIAL SYMBOL, Value: #
Type: KEYWORD, Value: include
Type: OPERATOR, Value: <
Type: IDENTIFIER, Value: iostream
Type: OPERATOR, Value: >
Type: KEYWORD, Value: using
Type: KEYWORD, Value: namespace
Type: IDENTIFIER, Value: std
Type: SPECIAL SYMBOL, Value: ;
Type: KEYWORD, Value: int
Type: IDENTIFIER, Value: factorial
Type: SPECIAL SYMBOL, Value: (
Type: KEYWORD, Value: int
Type: IDENTIFIER, Value: n
Type: SPECIAL SYMBOL, Value: )
Type: PUNCTUATOR, Value: {
Type: KEYWORD, Value: if
Type: SPECIAL SYMBOL, Value: (
Type: IDENTIFIER, Value: n
Type: IDENTIFIER, Value: <=
Type: IDENTIFIER, Value: 1
Type: SPECIAL SYMBOL, Value: )
Type: KEYWORD, Value: return
Type: IDENTIFIER, Value: 1
Type: SPECIAL SYMBOL, Value: ;
Type: KEYWORD, Value: return
Type: IDENTIFIER, Value: n
Type: IDENTIFIER, Value: *
Type: IDENTIFIER, Value: factorial
Type: SPECIAL SYMBOL, Value: (
Type: IDENTIFIER, Value: n
Type: OPERATOR, Value: -
Type: IDENTIFIER, Value: 1
Type: SPECIAL SYMBOL, Value: )
Type: SPECIAL SYMBOL, Value: ;
Type: PUNCTUATOR, Value: }
PS C:\Users\vishw\Coding\Compiler-Lab> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-1\" ; if ($?) { g++ LA.cpp -o LA } ; if ($?) { .\LA }
Type: KEYWORD, Value: int
Type: IDENTIFIER, Value: main
Type: SPECIAL SYMBOL, Value: (
Type: SPECIAL SYMBOL, Value: )
Type: PUNCTUATOR, Value: {
Type: KEYWORD, Value: int
Type: IDENTIFIER, Value: n
Type: SPECIAL SYMBOL, Value: ;
Type: IDENTIFIER, Value: cin
Type: OPERATOR, Value: >>
Type: IDENTIFIER, Value: n
Type: SPECIAL SYMBOL, Value: ;
Type: IDENTIFIER, Value: cout
Type: OPERATOR, Value: <<
Type: IDENTIFIER, Value: factorial
Type: SPECIAL SYMBOL, Value: (
Type: IDENTIFIER, Value: n
Type: SPECIAL SYMBOL, Value: )
Type: OPERATOR, Value: <<
Type: IDENTIFIER, Value: endl
Type: SPECIAL SYMBOL, Value: ;
Type: KEYWORD, Value: return
Type: IDENTIFIER, Value: 0
Type: SPECIAL SYMBOL, Value: ;
Type: PUNCTUATOR, Value: }
PS C:\Users\vishw\Coding\Compiler-Lab\Week-1>
```

Type: SPECIAL SYMBOL, Value: #
Type: KEYWORD, Value: include
Type: OPERATOR, Value: <
Type: IDENTIFIER, Value: iostream
Type: OPERATOR, Value: >
Type: KEYWORD, Value: using
Type: KEYWORD, Value: namespace

Type: IDENTIFIER, Value: std
 Type: SPECIAL SYMBOL, Value: ;
 Type: KEYWORD, Value: int
 Type: IDENTIFIER, Value: factorial
 Type: SPECIAL SYMBOL, Value: (
 Type: KEYWORD, Value: int
 Type: IDENTIFIER, Value: n
 Type: SPECIAL SYMBOL, Value:)
 Type: PUNCTUATOR, Value: {
 Type: KEYWORD, Value: if
 Type: SPECIAL SYMBOL, Value: (
 Type: IDENTIFIER, Value: n
 Type: IDENTIFIER, Value: <=
 Type: IDENTIFIER, Value: 1
 Type: SPECIAL SYMBOL, Value:)
 Type: KEYWORD, Value: return
 Type: IDENTIFIER, Value: 1
 Type: SPECIAL SYMBOL, Value: ;
 Type: KEYWORD, Value: return
 Type: IDENTIFIER, Value: n
 Type: IDENTIFIER, Value: *
 Type: IDENTIFIER, Value: factorial
 Type: SPECIAL SYMBOL, Value: (
 Type: IDENTIFIER, Value: n
 Type: OPERATOR, Value: -
 Type: IDENTIFIER, Value: 1
 Type: SPECIAL SYMBOL, Value:)
 Type: SPECIAL SYMBOL, Value: ;
 Type: PUNCTUATOR, Value: }
 Type: KEYWORD, Value: int
 Type: IDENTIFIER, Value: main
 Type: SPECIAL SYMBOL, Value: (
 Type: SPECIAL SYMBOL, Value:)
 Type: PUNCTUATOR, Value: {
 Type: KEYWORD, Value: int
 Type: IDENTIFIER, Value: n
 Type: SPECIAL SYMBOL, Value: ;
 Type: IDENTIFIER, Value: cin
 Type: OPERATOR, Value: >>
 Type: IDENTIFIER, Value: n
 Type: SPECIAL SYMBOL, Value: ;
 Type: IDENTIFIER, Value: cout
 Type: OPERATOR, Value: <<
 Type: IDENTIFIER, Value: factorial

Type: SPECIAL SYMBOL, Value: (
Type: IDENTIFIER, Value: n
Type: SPECIAL SYMBOL, Value:)
Type: OPERATOR, Value: <<
Type: IDENTIFIER, Value: endl
Type: SPECIAL SYMBOL, Value: ;
Type: KEYWORD, Value: return
Type: IDENTIFIER, Value: 0
Type: SPECIAL SYMBOL, Value: ;
Type: PUNCTUATOR, Value: }

Result:

The C++ program to perform lexical analysis of a factorial program was successfully run and the results were verified.

Lab Assignment

Name	S Vishwajith
Registration Number	23BCE1145
Subject	Compiler Design
Assignment Number	2

Question 1:

Write a regex to check if a string contains only alphabets.

Code:

```
// A C++ program to write a regex to check if a string contains only alphabets.
#include <iostream>
#include <regex>
using namespace std;

int main()
{
    string str;
    cout << "Enter a string: ";
    cin >> str;

    regex pattern("^[A-Za-z]+$");

    if (regex_match(str, pattern))
    {
        cout << "The string contains only alphabets." << endl;
    }
    else
    {
        cout << "The string contains characters other than alphabets." << endl;
    }

    return 0;
}
```

Output:

```
PS C:\Users\vishw\Coding\Compiler-Lab> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q1.cpp -o Q1 } ; if ($?) { .\Q1 }
Enter a string: abcdefg
The string contains only alphabets.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q1.cpp -o Q1 } ; if ($?) { .\Q1 }
Enter a string: abc1
The string contains characters other than alphabets.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> |
```

Question 2:

Write a regex to validate a 10-digit Indian mobile number starting with digits 6 to 9.

Code:

// A C++ code to write a regex to validate a 10-digit Indian mobile number starting with digits 6 to 9.

```
#include <iostream>
#include <regex>
using namespace std;

int main()
{
    string mobileNo;
    cout << "Enter a 10-digit Indian mobile number: ";
    cin >> mobileNo;
    regex pattern("^([6-9])[0-9]{9}$");
    if (regex_match(mobileNo, pattern))
    {
        cout << "The mobile number is valid." << endl;
    }
    else
    {
        cout << "The mobile number is invalid." << endl;
    }
}
```

Output:

```
PS C:\Users\vishw\Coding\Compiler-Lab> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q2.cpp -o Q2 } ; if ($?) { .\Q2 }
Enter a 10-digit Indian mobile number: 1234567890
The mobile number is invalid.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q2.cpp -o Q2 } ; if ($?) { .\Q2 }
Enter a 10-digit Indian mobile number: 6769686667
The mobile number is valid.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> |
```

Question 3:

Write a regex to validate an email address.

Code:

```
// A C++ program to write a regex to validate an email address.
#include <iostream>
#include <regex>
using namespace std;

int main()
{
    string emailAddress;
    cout << "Enter an email address: ";
    cin >> emailAddress;

    regex ep("^[a-zA-Z]+([a-zA-Z0-9_-.]*)@([a-zA-Z0-9-]+)\\.([a-zA-Z]{2,5})$");

    if (regex_match(emailAddress, ep))
    {
        cout << "This is a valid email address." << endl;
    }
    else
    {
        cout << "This is an invalid email address." << endl;
    }
    return 0;
}
```

Output:

```
PS C:\Users\vishw\Coding\Compiler-Lab> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q3.cpp -o Q3 } ; if ($?) { .\Q3 }
Enter an email address: something.something@something.something
This is an invalid email address.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q3.cpp -o Q3 } ; if ($?) { .\Q3 }
Enter an email address: vishwajith.s2023@gmail.com
This is a valid email address.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> |
```

Question 4:

Write a regex to validate dates in DD-MM-YYYY format.

Code:

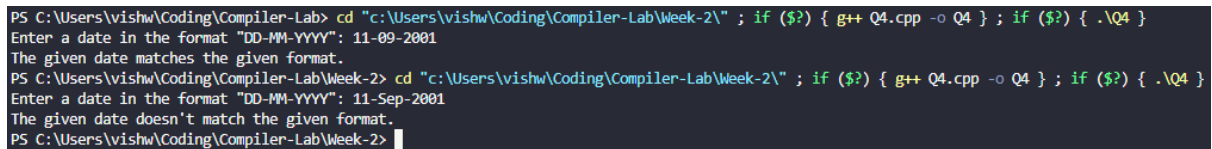
```
// A C++ program to write a regex to validate dates in DD-MM-YYYY format.
#include <iostream>
#include <regex>
using namespace std;

int main()
{
    string date;
    cout << "Enter a date in the format \"DD-MM-YYYY\": ";
    cin >> date;

    regex dp("^[0-3][0-9]\\-[0-1][0-9]\\-[0-9]{4}$");

    if (regex_match(date, dp))
    {
        cout << "The given date matches the given format." << endl;
    }
    else
    {
        cout << "The given date doesn't match the given format." << endl;
    }
    return 0;
}
```

Output:



```
PS C:\Users\vishw\Coding\Compiler-Lab> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q4.cpp -o Q4 } ; if ($?) { .\Q4 }
Enter a date in the format "DD-MM-YYYY": 11-09-2001
The given date matches the given format.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q4.cpp -o Q4 } ; if ($?) { .\Q4 }
Enter a date in the format "DD-MM-YYYY": 11-Sep-2001
The given date doesn't match the given format.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> |
```

Question 5:

Write a C++ program to check whether the string is accepted or not – the string must contain exactly two occurrences of 'a'.

Code:

```
// A C++ program to check whether the string is accepted or not – the string must contain
exactly two occurrences of 'a'.
#include <iostream>
```

```

#include <regex>
using namespace std;

int main()
{
    string s;
    cout << "Enter a string: ";
    cin >> s;

    regex sp("^a{2}$");
    if (regex_match(s, sp))
    {
        cout << "The string has exactly 2 occurrences of a." << endl;
    }
    else
    {
        cout << "The string doesn't have exactly 2 occurrences of a." << endl;
    }
    return 0;
}

```

Output:

```

PS C:\Users\vishw\Coding\Compiler-Lab> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q5.cpp -o Q5 } ; if ($?) { .\Q5 }
Enter a string: aa
The string has exactly 2 occurrences of a.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q5.cpp -o Q5 } ; if ($?) { .\Q5 }
Enter a string: ab
The string doesn't have exactly 2 occurrences of a.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q5.cpp -o Q5 } ; if ($?) { .\Q5 }
Enter a string: aaa
The string doesn't have exactly 2 occurrences of a.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2>

```

Question 6:

Write a C++ program to check whether the string is accepted or not – the string should be of even length and contain only a and b.

Code:

```

// A C++ program to check whether the string is accepted or not – the string should be of even
length and contain only a and b.
#include <iostream>
#include <regex>
using namespace std;

int main()

```

```

{
    string s;
    cout << "Enter a string: ";
    cin >> s;

    regex sp("^[a|b]+$");
    if (regex_match(s, sp) && s.size() % 2 == 0)
    {
        cout << "The string is of even length and contains only a and b." << endl;
    }
    else
    {
        cout << "The string doesn't satisfy one or more of the given conditions." << endl;
    }
    return 0;
}

```

Output:

```

PS C:\Users\vishw\Coding\Compiler-Lab> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q6.cpp -o Q6 } ; if ($?) { .\Q6 }
Enter a string: aabbabab
The string is of even length and contains only a and b.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q6.cpp -o Q6 } ; if ($?) { .\Q6 }
Enter a string: aab
The string doesn't satisfy one or more of the given conditions.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q6.cpp -o Q6 } ; if ($?) { .\Q6 }
Enter a string: bacd
The string doesn't satisfy one or more of the given conditions.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2>

```

Question 7:

Write a C++ program to check whether the string is accepted or not – the string should begin and end with the same character.

Code:

// A C++ program to check whether the string is accepted or not – the string should begin and end with the same character.

```

#include <iostream>
using namespace std;

```

```

int main()
{
    string s;
    cout << "Enter a string: ";
    cin >> s;
    if (s[0] == s[s.size() - 1])

```

```

{
    cout << "The first and last characters are the same." << endl;
}
else
{
    cout << "The first and last characters are not the same." << endl;
}
return 0;
}

```

Output:

```

PS C:\Users\vishw\Coding\Compiler-Lab> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q7.cpp -o Q7 } ; if ($?) { .\Q7 }
Enter a string: river
The first and last characters are the same.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q7.cpp -o Q7 } ; if ($?) { .\Q7 }
Enter a string: tamil
The first and last characters are not the same.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2>

```

Question 8:

Write a C++ program to check whether the string is accepted or not – the string must end with “abb”.

Code:

// A C++ program to check whether the string is accepted or not – the string must end with “abb”.

```

#include <iostream>
using namespace std;

```

```

int main()
{
    string s;
    cout << "Enter a string: ";
    cin >> s;
    if (s.size() >= 3 && s.substr(s.size() - 3) == "abb")
    {
        cout << "The string ends with \"abb\"." << endl;
    }
    else
    {
        cout << "The string does not end with \"abb\"." << endl;
    }
    return 0;
}

```


}

Output:

```
PS C:\Users\vishw\Coding\Compiler-Lab> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q8.cpp -o Q8 } ; if ($?) { .\Q8 }
Enter a string: sabb
The string ends with "abb".
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q8.cpp -o Q8 } ; if ($?) { .\Q8 }
Enter a string: abb
The string ends with "abb".
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q8.cpp -o Q8 } ; if ($?) { .\Q8 }
Enter a string: rapper
The string does not end with "abb".
```

Question 9:

Write a C++ program to check whether the string is accepted or not – the string must contain no consecutive repeating characters.

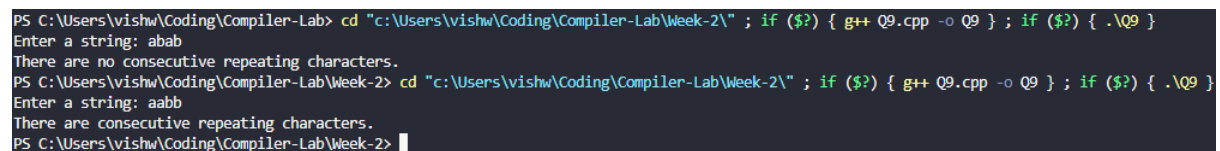
Code:

// A C++ program to check whether the string is accepted or not – the string must contain no consecutive repeating characters.

```
#include <iostream>
using namespace std;

int main()
{
    string s;
    cout << "Enter a string: ";
    cin >> s;
    bool flag = true;
    for (int i = 0; i < s.size() - 1; i++)
    {
        if (s[i] == s[i + 1])
        {
            flag = false;
            break;
        }
    }
    if (flag)
    {
        cout << "There are no consecutive repeating characters." << endl;
    }
    else
    {
        cout << "There are consecutive repeating characters." << endl;
    }
    return 0;
}
```

Output:



```
PS C:\Users\vishw\Coding\Compiler-Lab> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q9.cpp -o Q9 } ; if ($?) { .\Q9 }
Enter a string: abab
There are no consecutive repeating characters.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q9.cpp -o Q9 } ; if ($?) { .\Q9 }
Enter a string: aabb
There are consecutive repeating characters.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> |
```

Question 10:

Write a C++ program to check whether the string is accepted or not – the string must contain both substrings “ab” and “ba” at least once, in any order.

Code:

// a C++ program to check whether the string is accepted or not – the string must contain both substrings “ab” and “ba” at least once, in any order.

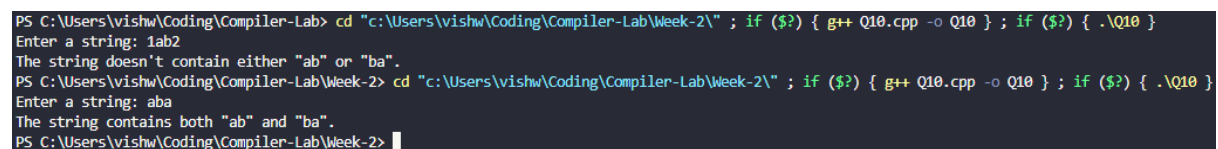
```
#include <iostream>
#include <regex>
using namespace std;

int main()
{
    string s;
    cout << "Enter a string: ";
    cin >> s;

    regex sp1("ab"), sp2("ba");

    if (regex_search(s, sp1) && regex_search(s, sp2))
    {
        cout << "The string contains both \"ab\" and \"ba\"." << endl;
    }
    else
    {
        cout << "The string doesn't contain either \"ab\" or \"ba\"." << endl;
    }
    return 0;
}
```

Output:



```
PS C:\Users\vishw\Coding\Compiler-Lab> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q10.cpp -o Q10 } ; if ($?) { .\Q10 }
Enter a string: 1ab2
The string doesn't contain either "ab" or "ba".
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q10.cpp -o Q10 } ; if ($?) { .\Q10 }
Enter a string: aba
The string contains both "ab" and "ba".
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2>
```

Question 11:

Write a C++ program to check whether the string is accepted or not – the string length must be a prime number, and the string must contain only 0s and 1s.

Code:

// A C++ program to check whether the string is accepted or not – the string length must be a prime number, and the string must contain only 0s and 1s.

```
#include <iostream>
```

```
#include <regex>
```

```
#include <math.h>
```

```
using namespace std;
```

```
bool isPrime(int n)
```

```
{
```

```
    if (n <= 1)
```

```
    {
```

```
        return false;
```

```
    }
```

```
    for (int i = 2; i <= sqrt(n); i++)
```

```
    {
```

```
        if (n % i == 0)
```

```
        {
```

```
            return false;
```

```
        }
```

```
    }
```

```
    return true;
```

```
}
```

```
int main()
```

```
{
```

```
    string s;
```

```
    cout << "Enter a string: ";
```

```
    cin >> s;
```

```
    regex sp("^[0|1]+$");
```

```
    if (regex_match(s, sp) && isPrime(s.size()))
```

```
    {
```

```
        cout << "The string is of prime number length and contains only 0 and 1." << endl;
```

```
    }
```

```
    else
```

```
    {
```

```
        cout << "The string doesn't satisfy one or more of the given conditions." << endl;
```

```
    }
```

```
    return 0;
```

```
}
```

Output:

```
PS C:\Users\vishw\Coding\Compiler-Lab> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q11.cpp -o Q11 } ; if ($?) { .\Q11 }
Enter a string: 01001
The string is of prime number length and contains only 0 and 1.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q11.cpp -o Q11 } ; if ($?) { .\Q11 }
Enter a string: 01aab
The string doesn't satisfy one or more of the given conditions.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-2\" ; if ($?) { g++ Q11.cpp -o Q11 } ; if ($?) { .\Q11 }
Enter a string: 0101
The string doesn't satisfy one or more of the given conditions.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-2> |
```

Compiler Design Lab

Date: 18-12-2025
Assignment No.: 3

Name: S Vishwajith
Register No.: 23BCE1145

Aim:

To write a C++ code to find Nullable(), Firstpos(), Lastpos() and Followpos() for the given Regular Expressions: $(a|b)^*a(a|b)(a|b)$, $(a|b)^*a(a|b)^*$, $(a|b)^*|(ac)^*$.

Algorithm:

1. Concatenate the Regular Expression with '#' for the end marker.
2. Construct a syntax tree.
3. Calculate the Nullable(), Firstpos(), Lastpos() and Followpos() using the rules for leaf nodes, Star (*) nodes, Concatenation (.) nodes and Or (|) nodes.
4. Maintain a vector of the number of characters that are not operators and print their Followpos().

Code:

RE_to_DFA.cpp:

```
#include <iostream>
#include <vector>
#include <string>
#include <set>
#include <stack>
#include <algorithm>
using namespace std;

string infix_to_postfix(string infix)
{
    string pf = "";
    stack<char> op_stack;
    for (char c : infix)
    {
        if (c != '*' && c != '|' && c != '.' && c != '(' && c != ')')
        {
            pf += c;
        }
    }
}
```

```

    }
    else if (c == '(')
    {
        op_stack.push(c);
    }
    else if (c == ')')
    {
        while (!op_stack.empty() && op_stack.top() != '(')
        {
            pf += op_stack.top();
            op_stack.pop();
        }
        op_stack.pop();
    }
    else
    {
        while (!op_stack.empty() &&
            ((c == '*' && (op_stack.top() == '*')) ||
             (c == '.' && (op_stack.top() == '*' || op_stack.top() == '.')) ||
             (c == '|' && (op_stack.top() == '*' || op_stack.top() == '.' || op_stack.top() == '|'))))
        {
            pf += op_stack.top();
            op_stack.pop();
        }
        op_stack.push(c);
    }
}
while (!op_stack.empty())
{
    pf += op_stack.top();
    op_stack.pop();
}
return pf;
}

```

```

vector<int> operator+(vector<int> a, vector<int> b)
{
    a.insert(a.end(), b.begin(), b.end());
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    return a;
}

```

class *tree*

```

{
public:
    char c;
    int lno;
    vector<int> first, last;
    bool nullable;
    tree *left;
    tree *right;

    tree(char ch, int n)
    {
        c = ch;
        lno = n;
        nullable = false;
        left = nullptr;
        right = nullptr;
        first.clear();
        last.clear();
    }
};

int main()
{
    string rx;
    cout << "Enter a Regular Expression in infix form: ";
    cin >> rx;
    rx = "(" + rx + ").#";
    string postfix = infix_to_postfix(rx);
    cout << "Postfix Expression: " << postfix << endl
        << endl;

    vector<vector<int>> follow;
    vector<char> chars;
    set<char> inputs;
    stack<tree *> st;
    int leafno = 0;
    tree *temp;

    for (int i = 0; i < postfix.size(); i++)
    {
        char t = postfix[i];
        if (t != '*' && t != '.' && t != '|')
        {
            cout << "Character at leaf node number " << leafno << " is: " << t << ". ";

```



```

chars.push_back(t);
if (t != '#')
    inputs.insert(t);
temp = new tree(t, leafno);
temp->first.push_back(leafno);
temp->last.push_back(leafno);

follow.push_back(vector<int>());
cout << "This node isn't nullable." << endl;
cout << "Firstpos: {" << leafno << "}" << endl;
cout << "Lastpos: {" << leafno << "}" << endl
    << endl;

leafno++;
}
else if (t == '*')
{
    cout << "This isn't a leaf node. It contains the character: '*' (star). It is nullable." << endl;
    temp = new tree(t, -1);
    temp->left = st.top();
    st.pop();
    temp->first = temp->left->first;
    temp->last = temp->left->last;
    temp->nullable = true;
    cout << "Firstpos: {";
    for (int j = 0; j < temp->first.size(); j++)
    {
        cout << temp->first[j] << ", ";
    }
    cout << "}" << endl;
    cout << "Lastpos: {";
    for (int j = 0; j < temp->last.size(); j++)
    {
        cout << temp->last[j] << ", ";
        follow[temp->last[j]] = follow[temp->last[j]] + temp->first;
    }
    cout << "}" << endl
        << endl;
}
else if (t == '.')
{
    cout << "This isn't a leaf node. It contains the character: '.' (concatenation). ";
    temp = new tree(t, -1);
    temp->right = st.top();

```

```

st.pop();
temp->left = st.top();
st.pop();
temp->nullable = temp->left->nullable && temp->right->nullable;
if (temp->nullable)
    cout << "The node is nullable." << endl;
else
    cout << "The node isn't nullable." << endl;

if (temp->left->nullable)
    temp->first = temp->left->first + temp->right->first;
else
    temp->first = temp->left->first;
cout << "Firstpos: {";
for (int j = 0; j < temp->first.size(); j++)
{
    cout << temp->first[j] << ", ";
}
cout << "}." << endl;

if (temp->right->nullable)
    temp->last = temp->right->last + temp->left->last;
else
    temp->last = temp->right->last;
cout << "Lastpos: {";
for (int j = 0; j < temp->last.size(); j++)
{
    cout << temp->last[j] << ", ";
}
cout << "}." << endl
    << endl;

for (int j = 0; j < temp->left->last.size(); j++)
{
    follow[temp->left->last[j]] = follow[temp->left->last[j]] + temp->right->first;
}
}
else if (t == '|')
{
    cout << "This isn't a leaf node. It contains the character: '|' (or). ";
    temp = new tree(t, -1);
    temp->right = st.top();
    st.pop();
    temp->left = st.top();

```

```

    st.pop();
    temp->nullable = temp->left->nullable || temp->right->nullable;
    if (temp->nullable)
        cout << "The node is nullable." << endl;
    else
        cout << "The node isn't nullable." << endl;

    temp->first = temp->left->first + temp->right->first;
    cout << "Firstpos: {";
    for (int j = 0; j < temp->first.size(); j++)
    {
        cout << temp->first[j] << ", ";
    }
    cout << "}." << endl;

    temp->last = temp->right->last + temp->left->last;
    cout << "Lastpos: {";
    for (int j = 0; j < temp->last.size(); j++)
    {
        cout << temp->last[j] << ", ";
    }
    cout << "}." << endl
        << endl;
}
st.push(temp);
}
cout << endl;
for (int i = 0; i < leafno; i++)
{
    cout << "Leaf number: " << i << ". ";
    cout << "Character: " << chars[i] << "." << endl;
    cout << "Followpos: {";
    for (int j = 0; j < follow[i].size(); j++)
        cout << follow[i][j] << ", ";
    cout << "}." << endl
        << endl;
}
return 0;
}

```

Inputs:

1. $(a|b)^*a(a|b)(a|b)$
2. $(a|b)^*a(a|b)^*$
3. $(a|b)^*|(ac)^*$

Output:

Enter a Regular Expression in infix form: $(a|b)^*.a.(a|b).(a|b)$

Postfix Expression: $ab|^*a.ab|.ab|. \#$.

Character at leaf node number 0 is: a. This node isn't nullable.

Firstpos: {0}.

Lastpos: {0}.

Character at leaf node number 1 is: b. This node isn't nullable.

Firstpos: {1}.

Lastpos: {1}.

This isn't a leaf node. It contains the character: '|' (or). The node isn't nullable.

Firstpos: {0, 1, }.

Lastpos: {0, 1, }.

This isn't a leaf node. It contains the character: '*' (star). It is nullable.

Firstpos: {0, 1, }.

Lastpos: {0, 1, }.

Character at leaf node number 2 is: a. This node isn't nullable.

Firstpos: {2}.

Lastpos: {2}.

This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.

Firstpos: {0, 1, 2, }.

Lastpos: {2, }.

Character at leaf node number 3 is: a. This node isn't nullable.

Firstpos: {3}.

Lastpos: {3}.

Character at leaf node number 4 is: b. This node isn't nullable.

Firstpos: {4}.

Lastpos: {4}.

This isn't a leaf node. It contains the character: '|' (or). The node isn't nullable.

Firstpos: {3, 4, }.

Lastpos: {3, 4, }.

This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.

Firstpos: {0, 1, 2, }.

Lastpos: {3, 4, }.

Character at leaf node number 5 is: a. This node isn't nullable.

Firstpos: {5}.

Lastpos: {5}.

Character at leaf node number 6 is: b. This node isn't nullable.

Firstpos: {6}.

Lastpos: {6}.

This isn't a leaf node. It contains the character: '|' (or). The node isn't nullable.

Firstpos: {5, 6, }.

Lastpos: {5, 6, }.

This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.

Firstpos: {0, 1, 2, }.

Lastpos: {5, 6, }.

Character at leaf node number 7 is: #. This node isn't nullable.

Firstpos: {7}.

Lastpos: {7}.

This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.

Firstpos: {0, 1, 2, }.

Lastpos: {7, }.

Leaf number: 0. Character: a.

Followpos: {0, 1, 2, }.

Leaf number: 1. Character: b.

Followpos: {0, 1, 2, }.

Leaf number: 2. Character: a.

Followpos: {3, 4, }.

Leaf number: 3. Character: a.

Followpos: {5, 6, }.

Leaf number: 4. Character: b.
Followpos: {5, 6, }.

Leaf number: 5. Character: a.
Followpos: {7, }.

Leaf number: 6. Character: b.
Followpos: {7, }.

Leaf number: 7. Character: #.
Followpos: {}.

```
Windows PowerShell
PS C:\Users\vishw> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-3&4" ; if ($?) { g++ RE_to_DFA.cpp -o RE_to_DFA } ; if ($?) { .\RE_to_DFA }
Enter a Regular Expression in infix form: (a|b)*.a.(a|b)*
Postfix Expression: ab|*a.ab|.a|.b|.
Character at leaf node number 0 is: a. This node isn't nullable.
Firstpos: {0}.
Lastpos: {0}.
Character at leaf node number 1 is: b. This node isn't nullable.
Firstpos: {1}.
Lastpos: {1}.
This isn't a leaf node. It contains the character: '|' (or). The node isn't nullable.
Firstpos: {0, 1}.
Lastpos: {0, 1}.
This isn't a leaf node. It contains the character: '*' (star). It is nullable.
Firstpos: {0, 1}.
Lastpos: {0, 1}.
Character at leaf node number 2 is: a. This node isn't nullable.
Firstpos: {2}.
Lastpos: {2}.
This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.
Firstpos: {0, 1, 2}.
Lastpos: {2}.
Character at leaf node number 3 is: a. This node isn't nullable.
Firstpos: {3}.
Lastpos: {3}.
Character at leaf node number 4 is: b. This node isn't nullable.
Firstpos: {4}.
Lastpos: {4}.
This isn't a leaf node. It contains the character: '|' (or). The node isn't nullable.
Firstpos: {3, 4}.
Lastpos: {3, 4}.
This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.
Firstpos: {0, 1, 2}.
Lastpos: {3, 4}.
Character at leaf node number 5 is: a. This node isn't nullable.
Firstpos: {5}.
Lastpos: {5}.
Character at leaf node number 6 is: b. This node isn't nullable.
Firstpos: {6}.
Lastpos: {6}.
This isn't a leaf node. It contains the character: '|' (or). The node isn't nullable.
Firstpos: {5, 6}.
Lastpos: {5, 6}.
This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.
Firstpos: {0, 1, 2}.
Lastpos: {5, 6}.
Character at leaf node number 7 is: #. This node isn't nullable.
Firstpos: {7}.
Lastpos: {7}.
This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.
Firstpos: {0, 1, 2}.
Lastpos: {7}.
Leaf number: 0. Character: a.
Followpos: {0, 1, 2}.
Leaf number: 1. Character: b.
Followpos: {0, 1, 2}.
Leaf number: 2. Character: a.
Followpos: {3, 4}.
Leaf number: 3. Character: a.
Followpos: {5, 6}.
Leaf number: 4. Character: b.
Followpos: {5, 6}.
Leaf number: 5. Character: a.
Followpos: {7}.
Leaf number: 6. Character: b.
Followpos: {7}.
Leaf number: 7. Character: #.
Followpos: {}.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-3&4>
```

Enter a Regular Expression in infix form: (a|b)*.a.(a|b)*

Postfix Expression: ab|*a.ab|.a|.b|. #.

Character at leaf node number 0 is: a. This node isn't nullable.

Firstpos: {0}.

Lastpos: {0}.

Character at leaf node number 1 is: b. This node isn't nullable.

Firstpos: {1}.

Lastpos: {1}.

This isn't a leaf node. It contains the character: '|' (or). The node isn't nullable.

Firstpos: {0, 1, }.

Lastpos: {0, 1, }.

This isn't a leaf node. It contains the character: '*' (star). It is nullable.

Firstpos: {0, 1, }.

Lastpos: {0, 1, }.

Character at leaf node number 2 is: a. This node isn't nullable.

Firstpos: {2}.

Lastpos: {2}.

This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.

Firstpos: {0, 1, 2, }.

Lastpos: {2, }.

Character at leaf node number 3 is: a. This node isn't nullable.

Firstpos: {3}.

Lastpos: {3}.

Character at leaf node number 4 is: b. This node isn't nullable.

Firstpos: {4}.

Lastpos: {4}.

This isn't a leaf node. It contains the character: '|' (or). The node isn't nullable.

Firstpos: {3, 4, }.

Lastpos: {3, 4, }.

This isn't a leaf node. It contains the character: '*' (star). It is nullable.

Firstpos: {3, 4, }.

Lastpos: {3, 4, }.

This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.

Firstpos: {0, 1, 2, }.

Lastpos: {2, 3, 4, }.

Character at leaf node number 5 is: #. This node isn't nullable.

Firstpos: {5}.

Lastpos: {5}.

This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.

Firstpos: {0, 1, 2, }.

Lastpos: {5, }.

Leaf number: 0. Character: a.
Followpos: {0, 1, 2, }.

Leaf number: 1. Character: b.
Followpos: {0, 1, 2, }.

Leaf number: 2. Character: a.
Followpos: {3, 4, 5, }.

Leaf number: 3. Character: a.
Followpos: {3, 4, 5, }.

Leaf number: 4. Character: b.
Followpos: {3, 4, 5, }.

Leaf number: 5. Character: #.
Followpos: {}.

```
Windows PowerShell
PS C:\Windows\system32> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-3&4\" ; if ($?) { g++ RE_to_DFA.cpp -o RE_to_DFA } ; if ($?) { .\RE_to_DFA }
Enter a Regular Expression in infix form: (a|b)*.a.(a|b)*
Postfix Expression: ab|*a.ab|#.
Character at leaf node number 0 is: a. This node isn't nullable.
Firstpos: {0}.
Lastpos: {0}.
Character at leaf node number 1 is: b. This node isn't nullable.
Firstpos: {1}.
Lastpos: {1}.
This isn't a leaf node. It contains the character: '|' (or). The node isn't nullable.
Firstpos: {0, 1}.
Lastpos: {0, 1}.
This isn't a leaf node. It contains the character: '*' (star). It is nullable.
Firstpos: {0, 1}.
Lastpos: {0, 1}.
Character at leaf node number 2 is: a. This node isn't nullable.
Firstpos: {2}.
Lastpos: {2}.
This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.
Firstpos: {0, 1, 2}.
Lastpos: {2}.
Character at leaf node number 3 is: a. This node isn't nullable.
Firstpos: {3}.
Lastpos: {3}.
Character at leaf node number 4 is: b. This node isn't nullable.
Firstpos: {4}.
Lastpos: {4}.
This isn't a leaf node. It contains the character: '|' (or). The node isn't nullable.
Firstpos: {3, 4}.
Lastpos: {3, 4}.
This isn't a leaf node. It contains the character: '*' (star). It is nullable.
Firstpos: {3, 4}.
Lastpos: {3, 4}.
This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.
Firstpos: {0, 1, 2}.
Lastpos: {2, 3, 4}.
Character at leaf node number 5 is: #. This node isn't nullable.
Firstpos: {5}.
Lastpos: {5}.
This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.
Firstpos: {0, 1, 2}.
Lastpos: {5}.

Windows PowerShell
Leaf number: 0. Character: a.
Followpos: {0, 1, 2, }.

Leaf number: 1. Character: b.
Followpos: {0, 1, 2, }.

Leaf number: 2. Character: a.
Followpos: {3, 4, 5, }.

Leaf number: 3. Character: a.
Followpos: {3, 4, 5, }.

Leaf number: 4. Character: b.
Followpos: {3, 4, 5, }.

Leaf number: 5. Character: #.
Followpos: {}.

PS C:\Users\vishw\Coding\Compiler-Lab\Week-3&4>
```

Enter a Regular Expression in infix form: (a|b)*|(a.c)*

Postfix Expression: ab|*ac.*|#.

Character at leaf node number 0 is: a. This node isn't nullable.

Firstpos: {0}.

Lastpos: {0}.

Character at leaf node number 1 is: b. This node isn't nullable.

Firstpos: {1}.

Lastpos: {1}.

This isn't a leaf node. It contains the character: '|' (or). The node isn't nullable.

Firstpos: {0, 1, }.

Lastpos: {0, 1, }.

This isn't a leaf node. It contains the character: '*' (star). It is nullable.

Firstpos: {0, 1, }.

Lastpos: {0, 1, }.

Character at leaf node number 2 is: a. This node isn't nullable.

Firstpos: {2}.

Lastpos: {2}.

Character at leaf node number 3 is: c. This node isn't nullable.

Firstpos: {3}.

Lastpos: {3}.

This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.

Firstpos: {2, }.

Lastpos: {3, }.

This isn't a leaf node. It contains the character: '*' (star). It is nullable.

Firstpos: {2, }.

Lastpos: {3, }.

This isn't a leaf node. It contains the character: '|' (or). The node is nullable.

Firstpos: {0, 1, 2, }.

Lastpos: {0, 1, 3, }.

Character at leaf node number 4 is: #. This node isn't nullable.

Firstpos: {4}.

Lastpos: {4}.

This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.

Firstpos: {0, 1, 2, 4, }.

Lastpos: {4, }.

Leaf number: 0. Character: a.

Followpos: {0, 1, 4, }.

Leaf number: 1. Character: b.

Followpos: {0, 1, 4, }.

Leaf number: 2. Character: a.

Followpos: {3, }.

Leaf number: 3. Character: c.

Followpos: {2, 4, }.

Leaf number: 4. Character: #.

Followpos: {}.

```
Windows PowerShell
PS C:\Users\vishw> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-3&4\" ; if ($?) { g++ RE_to_DFA.cpp -o RE_to_DFA } ; if ($?) { .\RE_to_DFA }
Enter a Regular Expression in infix form: (a|b)|(a.c)*
Postfix Expression: ab|*ac.*|#.

Character at leaf node number 0 is: a. This node isn't nullable.
Firstpos: {0}.
Lastpos: {0}.

Character at leaf node number 1 is: b. This node isn't nullable.
Firstpos: {1}.
Lastpos: {1}.

This isn't a leaf node. It contains the character: '|' (or). The node isn't nullable.
Firstpos: {0, 1, }.
Lastpos: {0, 1, }.

This isn't a leaf node. It contains the character: '*' (star). It is nullable.
Firstpos: {0, 1, }.
Lastpos: {0, 1, }.

Character at leaf node number 2 is: a. This node isn't nullable.
Firstpos: {2}.
Lastpos: {2}.

Character at leaf node number 3 is: c. This node isn't nullable.
Firstpos: {3}.
Lastpos: {3}.

This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.
Firstpos: {2, }.
Lastpos: {3, }.

This isn't a leaf node. It contains the character: '*' (star). It is nullable.
Firstpos: {2, }.
Lastpos: {3, }.

This isn't a leaf node. It contains the character: '|' (or). The node is nullable.
Firstpos: {0, 1, 2, }.
Lastpos: {0, 1, 3, }.

Character at leaf node number 4 is: #. This node isn't nullable.
Firstpos: {4}.
Lastpos: {4}.

Character at leaf node number 3 is: c. This node isn't nullable.
Firstpos: {3}.
Lastpos: {3}.

This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.
Firstpos: {2, }.
Lastpos: {3, }.

This isn't a leaf node. It contains the character: '*' (star). It is nullable.
Firstpos: {2, }.
Lastpos: {3, }.

This isn't a leaf node. It contains the character: '|' (or). The node is nullable.
Firstpos: {0, 1, 2, }.
Lastpos: {0, 1, 3, }.

Character at leaf node number 4 is: #. This node isn't nullable.
Firstpos: {4}.
Lastpos: {4}.

This isn't a leaf node. It contains the character: '.' (concatenation). The node isn't nullable.
Firstpos: {0, 1, 2, 4, }.
Lastpos: {4, }.

Leaf number: 0. Character: a.
Followpos: {0, 1, 4, }.

Leaf number: 1. Character: b.
Followpos: {0, 1, 4, }.

Leaf number: 2. Character: a.
Followpos: {3, }.

Leaf number: 3. Character: c.
Followpos: {2, 4, }.

Leaf number: 4. Character: #.
Followpos: {}.

PS C:\Users\vishw\Coding\Compiler-Lab\Week-3&4>
```

Result:

The C++ program to find Nullable(), Firstpos(), Lastpos(), and Followpos() for the given Regular Expressions was successfully run and the results were verified.

Compiler Design Lab

Date: 19-12-2025

Assignment No.: 4

Name: S Vishwajith

Register No.: 23BCE1145

Aim:

To write a C++ program that converts the regular expression $(a|b)^*a(a|b)$ to NFA with epsilon-transition and check whether the given string is accepted or rejected by the NFA.

Algorithm:

1. For the regular expression $(a|b)^*a(a|b)$, construct a transition table for NFA with epsilon (null) transitions containing the current state, symbol for transition and the new state.
2. Initiate states = {0} where 0 is the initial state and take the null closure of states in the variable and store it in the states variable itself.
3. For each symbol 'c' in the string:
 - a. Take a new empty variable 'next_states' or clear it if it exists.
 - b. For each 'st' in states:
 - i. If there are any possible transitions, add them to 'next_states'
 - c. Take the null closure of 'next_state'.
 - d. Replace state with 'next_state'.
4. If the states set contains the final state number 3, then the given string is accepted. Else, the given string is rejected.

Code:

```
#include <iostream>
#include <string>
#include <set>
#include <vector>
#include <map>
using namespace std;

map<int, vector<pair<char, int>>> get_transition_table()
{
    map<int, vector<pair<char, int>>> tt;

    tt[0].push_back({'a', 0});
    tt[0].push_back({'b', 0});
```

```

    tt[0].push_back({'e', 1});
    tt[1].push_back({'a', 2});
    tt[2].push_back({'a', 3});
    tt[2].push_back({'b', 3});

    return tt;
}

int get_next_state(vector<pair<char, int>> transitions, char symbol)
{
    for (auto &[c, i] : transitions)
    {
        if (c == symbol)
            return i;
    }
    return -1;
}

void null_closure(set<int> &states, map<int, vector<pair<char, int>>> tt)
{
    bool changed;
    do
    {
        changed = false;
        set<int> snapshot = states;
        for (int s : snapshot)
        {
            int e = get_next_state(tt[s], 'e');
            if (e != -1 && !states.count(e))
            {
                states.insert(e);
                changed = true;
            }
        }
    } while (changed);
}

int main()
{
    string s;
    cout << "Enter a string: ";
    cin >> s;

    map<int, vector<pair<char, int>>> tt = get_transition_table();

```

```

cout << endl
    << "Current State\tSymbol\tNew State" << endl;
for (auto &[old_state, transitions] : tt)
{
    for (auto &[symbol, new_state] : transitions)
    {
        cout << old_state << "\t\t" << symbol << "\t\t" << new_state << endl;
    }
}
cout << endl;

set<int> states = {0};
null_closure(states, tt);
for (char c : s)
{
    set<int> next_states;
    for (int st : states)
    {
        int ns = get_next_state(tt[st], c);
        if (ns != -1)
            next_states.insert(ns);
    }

    states = next_states;
    null_closure(states, tt);
}

for (int i : states)
{
    if (i == 3)
    {
        cout << "The given string is accepted." << endl;
        return 0;
    }
}

cout << "The given string is rejected." << endl;
return 0;
}

```

Inputs:

1. aa

2. ab
3. abba

Outputs:

Enter a string: aa

Current State	Symbol	New State
0	a	0
0	b	0
0	e	1
1	a	2
2	a	3
2	b	3

The given string is accepted.

```
PS C:\Users\vishw\Coding\Compiler-Lab> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-3&4\" ; if ($?) { g++ RE_to_NFA.cpp -o RE_to_NFA } ; if ($?) { .\RE_to_NFA }
Enter a string: aa

Current State  Symbol  New State
0             a       0
0             b       0
0             e       1
1             a       2
2             a       3
2             b       3

The given string is accepted.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-3&4> 
```

Enter a string: ab

Current State	Symbol	New State
0	a	0
0	b	0
0	e	1
1	a	2
2	a	3
2	b	3

The given string is accepted.

```
PS C:\Users\vishw\Coding\Compiler-Lab> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-3&4\" ; if ($?) { g++ RE_to_NFA.cpp -o RE_to_NFA } ; if ($?) { .\RE_to_NFA }
Enter a string: ab

Current State  Symbol  New State
0             a       0
0             b       0
0             e       1
1             a       2
2             a       3
2             b       3

The given string is accepted.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-3&4> 
```

Enter a string: abba

Current State	Symbol	New State
---------------	--------	-----------

0	a	0
0	b	0
0	e	1
1	a	2
2	a	3
2	b	3

The given string is rejected.

```
PS C:\Users\vishw\Coding\Compiler-Lab> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-3&4\" ; if ($?) { g++ RE_to_NFA.cpp -o RE_to_NFA } ; if ($?) { .\RE_to_NFA }
Enter a string: abba

Current State  Symbol  New State
0             a           0
0             b           0
0             e           1
1             a           2
2             a           3
2             b           3

The given string is rejected.
PS C:\Users\vishw\Coding\Compiler-Lab\Week-3&4> 
```

Result:

The C++ program to convert the regex $(a|b)^*a(a|b)$ into NFA with epsilon transitions and check whether the given string is accepted or not was successfully run and the results were verified.

Compiler Design Lab

Date: 9-1-2026

Assignment No.: 5

Name: S Vishwajith

Register No.: 23BCE1145

Aim:

To write a C++ program that parses the given LL (1) grammar: $E \rightarrow E+T/T$; $T \rightarrow T^*F/F$; $F \rightarrow (E)/id$; and prints the first(), follow(), parse table and the stack trace, and check if the string "(id+id)*id" is accepted by the grammar or not.

Algorithm:

1. Eliminate left recursion present in the given grammar.
2. Find first() for all non-terminals.
3. Using first(), find follow() for all non-terminals.
4. Using first() and follow(), construct the parse table.
5. Using this parse table, check whether the string can be accepted or not, and print the trace for each step.

Code:

```
#include <iostream>
#include <vector>
#include <string>
#include <map>
#include <set>
#include <iomanip>
using namespace std;

vector<pair<char, string>> get_grammar()
{
    return {
        {'E', "TA"},
        {'A', "+TA"},
        {'A', "e"},
        {'T', "FB"},
        {'B', "*FB"},
        {'B', "e"},
        {'F', "(E)"},
    };
}
```

```

    {'F', "id"};
}

map<char, set<string>> FIRST, FOLLOW;
map<char, map<string, string>> TABLE;

bool isNonTerminal(char c)
{
    return c >= 'A' && c <= 'Z';
}

void computeFIRST(vector<pair<char, string>> &G)
{
    bool changed = true;
    while (changed)
    {
        changed = false;
        for (auto &prod : G)
        {
            char A = prod.first;
            string rhs = prod.second;

            if (rhs == "ε")
            {
                if (FIRST[A].insert("ε").second)
                    changed = true;
                continue;
            }

            for (int i = 0; i < rhs.length(); i++)
            {
                char X = rhs[i];

                if (isNonTerminal(X))
                {
                    for (auto t : FIRST[X])
                    {
                        if (t != "ε")
                            if (FIRST[A].insert(t).second)
                                changed = true;
                    }

                    if (FIRST[X].count("ε") == 0)
                        break;
                }
                else

```

```

    {
        if (X == 'i')
            FIRST[A].insert("id");
        else
            FIRST[A].insert(string(1, X));
        break;
    }
}
}
}
}

```

```

void computeFOLLOW(vector<pair<char, string>> &G)

```

```

{
    FOLLOW['E'].insert("$");

    bool changed = true;
    while (changed)
    {
        changed = false;
        for (auto &prod : G)
        {
            char A = prod.first;
            string rhs = prod.second;

            for (int i = 0; i < rhs.length(); i++)
            {
                char B = rhs[i];
                if (!isNonTerminal(B))
                    continue;

                bool epsilonTrail = true;

                for (int j = i + 1; j < rhs.length(); j++)
                {
                    epsilonTrail = false;
                    char C = rhs[j];

                    if (isNonTerminal(C))
                    {
                        for (auto t : FIRST[C])
                        {
                            if (t != "ε")
                                if (FOLLOW[B].insert(t).second)
                                    changed = true;

```

```

        if (FIRST[C].count("e"))
            epsilonTrail = true;
        else
            break;
    }
    else
    {
        if (C == 'i')
            FOLLOW[B].insert("id");
        else
            FOLLOW[B].insert(string(1, C));
        break;
    }
}
}

if (epsilonTrail || i == rhs.length() - 1)
    for (auto t : FOLLOW[A])
        if (FOLLOW[B].insert(t).second)
            changed = true;
    }
}
}
}

```

```

set<string> FIRST_of_string(string s)
{
    set<string> result;
    for (int i = 0; i < s.length(); i++)
    {
        char X = s[i];

        if (isNonTerminal(X))
        {
            for (auto t : FIRST[X])
                if (t != "e")
                    result.insert(t);
            if (FIRST[X].count("e") == 0)
                break;
        }
        else
        {
            if (X == 'i')
                result.insert("id");
        }
    }
}

```

```

        else
            result.insert(string(1, X));
        break;
    }
}
return result;
}

```

```

void buildParsingTable(vector<pair<char, string>> &G)
{
    for (auto &prod : G)
    {
        char A = prod.first;
        string alpha = prod.second;

        auto f = FIRST_of_string(alpha);

        for (auto t : f)
            TABLE[A][t] = alpha;

        if (f.count("ε"))
            for (auto b : FOLLOW[A])
                TABLE[A][b] = alpha;
    }
}

```

```

vector<string> tokenize(string s)
{
    vector<string> tokens;

    for (int i = 0; i < s.length(); i++)
    {
        if (i + 1 < s.length() && s.substr(i, 2) == "id")
        {
            tokens.push_back("id");
            i += 2;
        }
        else
        {
            tokens.push_back(string(1, s[i]));
            i++;
        }
    }
    return tokens;
}

```

```

}

int parse(string input)
{
    input += "$";

    vector<string> stk = {"$", "E"};
    int i = 0;

    cout << left
        << setw(20) << "Stack"
        << setw(20) << "Input"
        << "Action" << endl;
    cout << "-----" << endl;

    while (true)
    {
        string stackStr;
        for (auto &s : stk)
            stackStr += s;

        cout << left
            << setw(20) << stackStr
            << setw(20) << input.substr(i);

        string X = stk.back();

        if (X == "$" && input[i] == '$')
        {
            cout << "Accept\n";
            return 0;
        }

        string a;
        if (i + 1 < input.length() && input.substr(i, 2) == "id")
            a = "id";
        else
            a = string(1, input[i]);

        if (!isNonTerminal(X[0]))
        {
            if (X == a)
            {
                stk.pop_back();
            }
        }
    }
}

```



```

        i += (a == "id" ? 2 : 1);
        cout << "Match " << a << "\n";
    }
    else
    {
        cout << "Error\n";
        return 1;
    }
}
else
{
    if (TABLE[X[0]].count(a) == 0)
    {
        cout << "Error\n";
        return 1;
    }

    stk.pop_back();
    string prod = TABLE[X[0]][a];

    if (prod != "e")
    {
        auto tokens = tokenize(prod);
        for (int k = tokens.size() - 1; k >= 0; k--)
            stk.push_back(tokens[k]);
    }

    cout << X << " --> " << prod << "\n";
}
}
}

int main()
{
    auto grammar = get_grammar();

    computeFIRST(grammar);
    computeFOLLOW(grammar);
    buildParsingTable(grammar);

    cout << endl
        << "First():" << endl;
    for (auto &p : FIRST)
    {

```

```

        cout << "first(" << p.first << ") = { ";
        for (auto &x : p.second)
            cout << x << " ";
        cout << "}\n";
    }

    cout << endl
        << "Follow():" << endl;
    for (auto &p : FOLLOW)
    {
        cout << "follow(" << p.first << ") = { ";
        for (auto &x : p.second)
            cout << x << " ";
        cout << "}\n";
    }

    cout << endl
        << "Parsing Table:" << endl;
    for (auto &r : TABLE)
        for (auto &c : r.second)
            cout << "map[" << r.first << "," << c.first << "] = " << r.first << " --> " << c.second << endl;

    string input;
    cout << endl
        << "Enter input string: ";
    cin >> input;

    int res = parse(input);
    cout << endl
        << ((res == 0) ? "String is accepted." : "String is rejected.") << endl;
}

```

Input/Output:

First():

first(A) = { + e }

first(B) = { * e }

first(E) = { (id }

first(F) = { (id }

first(T) = { (id }

Follow():

follow(A) = { \$) }

follow(B) = { \$) + }

$\text{follow}(E) = \{ \$ \}$
 $\text{follow}(F) = \{ \$ \}^* + \}$
 $\text{follow}(T) = \{ \$ \} + \}$

Parsing Table:

$\text{map}[A, \$] = A \rightarrow e$
 $\text{map}[A,)] = A \rightarrow e$
 $\text{map}[A, +] = A \rightarrow +TA$
 $\text{map}[A, e] = A \rightarrow e$
 $\text{map}[B, \$] = B \rightarrow e$
 $\text{map}[B,)] = B \rightarrow e$
 $\text{map}[B, *] = B \rightarrow *FB$
 $\text{map}[B, +] = B \rightarrow e$
 $\text{map}[B, e] = B \rightarrow e$
 $\text{map}[E, (] = E \rightarrow TA$
 $\text{map}[E, id] = E \rightarrow TA$
 $\text{map}[F, (] = F \rightarrow (E)$
 $\text{map}[F, id] = F \rightarrow id$
 $\text{map}[T, (] = T \rightarrow FB$
 $\text{map}[T, id] = T \rightarrow FB$

Enter input string: (id+id)*id

Stack	Input	Action
\$E	(id+id)*id\$	E $\rightarrow$ TA
\$AT	(id+id)*id\$	T $\rightarrow$ FB
\$ABF	(id+id)*id\$	F $\rightarrow$ (E)
\$AB)E(	(id+id)*id\$	Match (
\$AB)E	id+id)*id\$	E $\rightarrow$ TA
\$AB)AT	id+id)*id\$	T $\rightarrow$ FB
\$AB)ABF	id+id)*id\$	F $\rightarrow$ id
\$AB)ABid	id+id)*id\$	Match id
\$AB)AB	+id)*id\$	B $\rightarrow$ e
\$AB)A	+id)*id\$	A $\rightarrow$ +TA
\$AB)AT+	+id)*id\$	Match +
\$AB)AT	id)*id\$	T $\rightarrow$ FB
\$AB)ABF	id)*id\$	F $\rightarrow$ id
\$AB)ABid	id)*id\$	Match id
\$AB)AB	)id\$	B $\rightarrow$ e
\$AB)A	)id\$	A $\rightarrow$ e
\$AB)	)id\$	Match)
\$AB	*id\$	B $\rightarrow$ *FB
\$ABF*	*id\$	Match *
\$ABF	id\$	F $\rightarrow$ id

\$ABid	id\$	Match id
\$AB	\$	B --> e
\$A	\$	A --> e
\$	\$	Accept

String is accepted.

```
Windows PowerShell
PS C:\Users\vishw> cd "c:\Users\vishw\Coding\Compiler-Lab\Week 5\" ; if ($?) { g++ LL_1_Parser.cpp -o LL_1_Parser } ; if ($?) { .\LL_1_Parser }

First():
first(A) = { + e }
first(B) = { * e }
first(E) = { ( id }
first(F) = { ( id }
first(T) = { ( id }

Follow():
follow(A) = { $ ) }
follow(B) = { $ ) + }
follow(E) = { $ ) }
follow(F) = { $ ) * + }
follow(T) = { $ ) + }

Parsing Table:
map[A,$] = A --> e
map[A,)] = A --> e
map[A,+ ] = A --> +TA
map[A,e] = A --> e
map[B,$] = B --> e
map[B,)] = B --> e
map[B,*] = B --> *FB
map[B,+ ] = B --> e
map[B,e] = B --> e
map[E,(] = E --> TA
map[E,id] = E --> TA
map[F,(] = F --> (E)
map[F,id] = F --> id
map[T,(] = T --> FB
map[T,id] = T --> FB

Enter input string: (id+id)*id

Enter input string: (id+id)*id
Stack      Input      Action
-----
$E         (id+id)*id$    E --> TA
$AT        (id+id)*id$    T --> FB
$ABF       (id+id)*id$    F --> (E)
$AB)E(     (id+id)*id$    Match (
$AB)E      id+id)*id$    E --> TA
$AB)AT     id+id)*id$    T --> FB
$AB)ABF    id+id)*id$    F --> id
$AB)ABid   id+id)*id$    Match id
$AB)AB     +id)*id$      B --> e
$AB)A      +id)*id$      A --> +TA
$AB)AT+    +id)*id$      Match +
$AB)AT     id)*id$      T --> FB
$AB)ABF    id)*id$      F --> id
$AB)ABid   id)*id$      Match id
$AB)AB     )*id$        B --> e
$AB)A      )*id$        A --> e
$AB)       )*id$        Match )
$AB        *id$        B --> *FB
$ABF*      *id$        Match *
$ABF       id$        F --> id
$ABid      id$        Match id
$AB        $         B --> e
$A         $         A --> e
$          $         Accept

String is accepted.
PS C:\Users\vishw\Coding\Compiler-Lab\Week 5> |
```

Result:

The C++ program that parses the given LL (1) grammar: $E \rightarrow E+T/T$; $T \rightarrow T*F/F$; $F \rightarrow (E)/id$; and prints the first(), follow(), parse table and the stack trace, and check if the string "(id+id)*id" is accepted by the grammar or not was successfully run and the results were verified.

Compiler Design Lab

Date: 23-1-2026
Assignment No.: 5

Name: S Vishwajith
Register No.: 23BCE1145

Aim:

To write a C++ program to construct the Leading() and Trailing() for the given grammar.

Algorithm:

1. Read the Grammar.
2. For each non-terminal A, find Leading() by analysing each of its production.
 - a. If the non-terminal produces a terminal first, add the terminal to its Leading().
 - b. If it produces a non-terminal B first and then a terminal, add terminal to its Leading() and if B is not A, add Leading(B) to Leading(A)
3. For each non-terminal A, find Trailing() by analysing each of its production.
 - a. If the non-terminal produces a terminal last, add the terminal to its Trailing().
 - b. If it produces a non-terminal B at the last and then a terminal, add terminal to its Trailing() and if B is not A, add Trailing(B) to Trailing(A).
4. Display the Leading() and Trailing() of each non-terminal.

Code:

```
#include <iostream>
#include <string>
#include <vector>
#include <set>
#include <map>
#include <stack>
#include <limits>
using namespace std;

struct NonTerminal
{
    char nt;
    vector<string> rhs;
    set<char> lead;
    set<char> trail;
};
```

```

vector<struct NonTerminal> p;

bool isNonTerminal(char c)
{
    for (struct NonTerminal temp_p : p)
    {
        if (temp_p.nt == c)
        {
            return true;
        }
    }
    return false;
}

set<char> getLeading(char nt)
{
    set<char> lead;
    for (struct NonTerminal temp_p : p)
    {
        if (temp_p.nt == nt)
        {
            for (string s : temp_p.rhs)
            {
                if (!isNonTerminal(s[0]))
                {
                    lead.insert(s[0]);
                }
                else if (s.length() > 1)
                {
                    if (!isNonTerminal(s[1]))
                    {
                        lead.insert(s[1]);
                    }
                    if (s[0] != temp_p.nt)
                    {
                        for (char c : getLeading(s[0]))
                        {
                            lead.insert(c);
                        }
                    }
                }
            }
        }
    }
}

```

```

    }
    return lead;
}

set<char> getTrailing(char nt)
{
    set<char> trail;
    for (struct NonTerminal temp_p : p)
    {
        if (temp_p.nt == nt)
        {
            for (string s : temp_p.rhs)
            {
                if (!isNonTerminal(s[s.length() - 1]))
                {
                    trail.insert(s[s.length() - 1]);
                }
                else if (s.length() > 1)
                {
                    if (!isNonTerminal(s[s.length() - 2]))
                    {
                        trail.insert(s[s.length() - 2]);
                    }
                    if (s[s.length() - 1] != temp_p.nt)
                    {
                        for (char c : getTrailing(s[s.length() - 1]))
                        {
                            trail.insert(c);
                        }
                    }
                }
            }
        }
    }
    return trail;
}

```

```

void createStruct(string s)
{
    struct NonTerminal temp;
    vector<string> temp_rhs;
    int i = 0;
    bool start_rhs = false;
    string temp_r = "", temp_l = "";

```



```

while (i < s.length())
{
    if (s[i] == ' ')
    {
        i++;
        continue;
    }
    if (start_rhs)
    {
        if (s[i] != '|')
        {
            temp_r += s[i];
        }
        else
        {
            temp_rhs.push_back(temp_r);
            temp_r = "";
        }
    }
    else
    {
        if (s[i] == '-' && s[i + 1] == '>')
        {
            start_rhs = true;
            i += 2;
        }
        else
            temp_l += s[i];
    }
    i++;
}
temp_rhs.push_back(temp_r);
temp.nt = temp_l[0];
temp.rhs = temp_rhs;
p.push_back(temp);
}

```

```

int main()
{
    int n;
    cout << "Enter number of productions: ";
    cin >> n;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    vector<string> temp_prod;

```

```

string input;
for (int i = 0; i < n; i++)
{
    cout << "Enter Production " << i + 1 << ": ";
    getline(cin, input);
    temp_prod.push_back(input);
}
for (int i = 0; i < n; i++)
{
    // cout<<temp_prod[i]<<endl;
    createStruct(temp_prod[i]);
}
cout << "Productions" << endl;
for (struct NonTerminal temp_p : p)
{
    cout << temp_p.nt << " -> ";
    for (int i = 0; i < temp_p.rhs.size(); i++)
    {
        cout << temp_p.rhs[i];
        if (i != temp_p.rhs.size() - 1)
        {
            cout << '|';
        }
    }
    cout << endl;
}
cout << "\nLeading:" << endl;
for (auto &temp_p : p)
{
    temp_p.lead = getLeading(temp_p.nt);
    cout << temp_p.nt << ":{" ";
    for (char c : temp_p.lead)
    {
        cout << c << " ";
    }
    cout << "}" << endl;
}
cout << "\nTrailing:" << endl;
for (auto &temp_p : p)
{
    temp_p.trail = getTrailing(temp_p.nt);
    cout << temp_p.nt << ":{" ";
    for (char c : temp_p.trail)
    {

```

```

        cout << c << " ";
    }
    cout << "}" << endl;
}
return 0;
}

```

Inputs:

1. $E \rightarrow EAE \mid (E) \mid id$; $A \rightarrow + \mid - \mid * \mid /$
2. $S \rightarrow (L) \mid a$; $L \rightarrow L,S \mid S$

Outputs:

1.

Enter number of productions: 2

Enter Production 1: $E \rightarrow EAE \mid (E) \mid id$

Enter Production 2: $A \rightarrow + \mid - \mid * \mid /$

Productions

$E \rightarrow EAE \mid (E) \mid id$

$A \rightarrow + \mid - \mid * \mid /$

Leading:

$E: \{ (\mid i \}$

$A: \{ * \mid + \mid - \mid / \}$

Trailing:

$E: \{) \mid d \}$

$A: \{ * \mid + \mid - \mid / \}$

```

PS C:\Users\vishw\Coding\Compiler-Lab> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-6\" ; if ($?) { g++ Leading_Trailing.cpp -o Leading_Trailing } ; if ($?) { .\Leading_Trailing }
Enter number of productions: 2
Enter Production 1: E -> EAE | (E) | id
Enter Production 2: A -> + | - | * | /
Productions
E -> EAE|(E)|id
A -> +|-|*|/

Leading:
E:{ ( i }
A:{ * + - / }

Trailing:
E:{ ) d }
A:{ * + - / }

```

2.

Enter number of productions: 2

Enter Production 1: $S \rightarrow (L) \mid a$

Enter Production 2: $L \rightarrow L,S \mid S$

Productions

$S \rightarrow (L) \mid a$

$L \rightarrow L,S \mid S$

Leading:

S:{ (a }

L:{ , }

Trailing:

S:{) a }

L:{) , a }

```
PS C:\Users\vishw\coding\Compiler-Lab\Week-6> cd "c:\Users\vishw\coding\Compiler-Lab\Week-6\" ; if ($?) { g++ Leading_Trailing.cpp -o Leading_Trailing } ; if ($?) { .\Leading_Trailing }
Enter number of productions: 2
Enter Production 1: S -> (L) | a
Enter Production 2: L -> L,S | S
Productions
S -> (L)|a
L -> L,S|S

Leading:
S:{ ( a }
L:{ , }

Trailing:
S:{ ) a }
L:{ ) , a }
```

Result:

The C++ program to construct the Leading() and Trailing() for the given grammar was successfully run and the results were verified.

Compiler Design Lab

Date: 6-2-2026

Assignment No.: 7

Name: S Vishwajith

Register No.: 23BCE1145

Aim:

To write the given programs in LEX:

1. Write a LEX program to count the number of characters in a given input.
2. Write a LEX program to count the number of words in a sentence.
3. Write a LEX program to count the number of lines in a file.
4. Write a LEX program to count the number of vowels in the given input.
5. Write a LEX program to convert all lowercase letters to uppercase.
6. Write a LEX program to identify and count digits in the input.
7. Write a LEX program to identify and count keywords and identifiers.
8. Write a LEX program to recognize integers and floating-point numbers.
9. Write a LEX program to count the number of operators and operands.
10. Write a LEX program to remove single-line and multi-line comments from a C program.
11. Write a LEX program to recognize relational and logical operators.
12. Write a LEX program to count the number of uppercase and lowercase letters.
13. Write a LEX program to identify valid and invalid identifiers.
14. Write a LEX program to find and print all email IDs from a given text.
15. Write a LEX program to perform lexical analysis of a C program.
16. Write a LEX program to generate tokens for keywords, identifiers, constants, and operators.
17. Write a LEX program to implement a symbol table.
18. Write a LEX program to validate arithmetic expression.

Codes:

```
1.
/* Write a LEX program to count the number of characters in a given input. */

%{
#include <stdio.h>
int characters = 0;
}%

%%
. {characters++;}
\n {characters++;}
```

```
%%
```

```
int yywrap(){  
    return 1;  
}
```

```
int main(){  
    printf("Input here: ");  
    yylex();  
    printf("The number of characters in the given input is: %d.\n", characters);  
    return 0;  
}
```

2.

/* Write a LEX program to count the number of words in a sentence. */

```
%{  
    #include <stdio.h>  
    int words = 0;  
}%
```

```
%%  
[a-zA-Z]+ {words++;}  
[ \n\t]+ {}  
. {}  
%%
```

```
int yywrap(){  
    return 1;  
}
```

```
int main(){  
    printf("Input here: ");  
    yylex();  
    printf("The number of words is: %d.\n", words);  
    return 0;  
}
```

3.

/* Write a LEX program to count the number of lines in a file. */

```
%{  
    #include <stdio.h>  
    int lines = 0;  
}%
```

```
%%
. {}
\n {lines++;}
%%
```

```
int yywrap() {
    return 1;
}
```

```
int main() {
    printf("Input here: ");
    yylex();
    printf("The number of lines is: %d.\n", lines);
    return 0;
}
```

4.

/* Write a LEX program to count the number of vowels in the given input. */

```
%{
    #include <stdio.h>
    int vowels = 0;
}%
```

```
%%
[aeiouAEIOU] {vowels++;}
.\n {}
%%
```

```
int yywrap() {
    return 1;
}
```

```
int main(){
    printf("Input here: ");
    yylex();
    printf("The number of vowels are: %d.\n", vowels);
    return 0;
}
```

5.

/* Write a LEX program to convert all lowercase letters to uppercase. */

```
%{
#include <stdio.h>
#include <ctype.h>
}%
```

```
%%
[a-z]    { yytext[0] = toupper(yytext[0]); ECHO; }
.|\\n    { ECHO; }
%%
```

```
int yywrap() {
    return 1;
}
```

```
int main() {
    printf("Input here: ");
    fflush(stdout);
    yylex();
    return 0;
}
```

6.

/* Write a LEX program to identify and count digits in the input. */

```
%{
    #include <stdio.h>
    int digits = 0;
}%
```

```
%%
[0-9] {digits++;}
.|\\n {}
%%
```

```
int yywrap() {
    return 1;
}
```

```
int main() {
    printf("Input here: ");
    yylex();
    printf("The number of digits is: %d.\\n", digits);
    return 0;
}
```

7.

/* Write a LEX program to identify and count keywords and identifiers. */

```
%{
    #include <stdio.h>
```



```

#include <string.h>
int keywords = 0;
int identifiers = 0;
int is_keyword(char* token) {
    char *keywords[] = {"auto", "break", "case", "char", "const", "continue",
        "default", "do", "double", "else", "enum", "extern",
        "float", "for", "goto", "if", "int", "long", "register",
        "return", "short", "signed", "sizeof", "static", "struct",
        "switch", "typedef", "union", "unsigned", "void", "volatile",
        "while"};

    int n = sizeof(keywords) / sizeof(char*);
    for(int i = 0; i < n; i++) {
        if(strcmp(token, keywords[i]) == 0) {
            return 1;
        }
    }
    return 0;
}
%}

```

```

identifier [a-zA-Z_][a-zA-Z0-9_]*
whitespace [ \t\n]+

```

```

%%
{identifier} {
    if(is_keyword(yytext)) {
        keywords++;
        printf("KEYWORD: %s.\n", yytext);
    }
    else {
        identifiers++;
        printf("IDENTIFIER: %s.\n", yytext);
    }
}
{whitespace} {}
. {}
%%

```

```

int yywrap() {
    return 1;
}

```

```

int main() {
    printf("Write code here:\n");
}

```



```
%%
```

```
int yywrap() {  
    return 1;  
}
```

```
int main() {  
    printf("Inputs here:\n");  
    yylex();  
    printf("Number of operands: %d.\n", operands);  
    printf("Number of operators: %d.\n", operators);  
    return 0;  
}
```

10.

/* Write a LEX program to remove single-line and multi-line comments from a C program. */

```
%{  
    #include <stdio.h>  
    #include <string.h>  
    #define MAX 100000  
    char code[MAX];  
    int x = 0;  
    void add_text(const char *text) {  
        int len = strlen(text);  
        if (x + len < MAX) {  
            strcpy(&code[x], text);  
            x += len;  
        }  
    }  
}%
```

```
%x SINGLE MULTI
```

```
%%
```

```
\("[^\\"]|\\.)*\" {add_text(yytext);}  
\'([^\']|\\.)*\' {add_text(yytext);}  
"/" {BEGIN(SINGLE);}  
"/*" {BEGIN(MULTI);}
```

```
<SINGLE>\n {BEGIN(INITIAL); add_text("\n");}  
<SINGLE>. {}
```

```
<MULTI>"/" {BEGIN(INITIAL);}  
<MULTI>\n {add_text("\n");}
```

```
<MULTI>. {}
```

```
.\n {add_text(yytext);}
%%
```

```
int yywrap() {
    return 1;
}
```

```
int main() {
    printf("Enter C Code:\n");
    yylex();
    printf("The C code without any comments is:\n\n%s\n", code);
    return 0;
}
```

11.

/* Write a LEX program to recognize relational and logical operators. */

```
%{
    #include <stdio.h>
}%
```

```
relational "<="|">="|"=="|"!="|"<"|>"
logical "&&"|"||"|"!"
```

```
%%
{relational} {printf("RELATIONAL OPERATOR: %s.\n", yytext);}
{logical} {printf("LOGICAL OPERATOR: %s.\n", yytext);}
.\n {}
%%
```

```
int yywrap() {
    return 1;
}
```

```
int main() {
    printf("Inputs here:\n");
    yylex();
    return 0;
}
```

12.

/* Write a LEX program to count the number of uppercase and lowercase letters. */

```
%{
```

```

#include <stdio.h>
int upper = 0;
int lower = 0;
%}

%%
[a-z] {lower++;}
[A-Z] {upper++;}
.|\n {}
%%

int yywrap() {
    return 1;
}

int main() {
    printf("Input here:");
    yylex();
    printf("Number of uppercase letters is: %d.\n", upper);
    printf("Number of lowercase letters is: %d.\n", lower);
    return 0;
}
13.
/* Write a LEX program to identify valid and invalid identifiers. */

%{
#include <stdio.h>
#include <string.h>
const char *keywords[] = {
    "int", "float", "char", "double", "if", "else", "while", "for",
    "return", "void", "break", "continue", "switch", "case", "default",
    "do", "struct", "union", "typedef", "enum", "const", "unsigned",
    "signed", "extern", "static", "sizeof", "long", "short", "goto"
};
int isKeyword(const char *word) {
    int n = sizeof(keywords)/sizeof(keywords[0]);
    for(int i = 0; i < n; i++) {
        if(strcmp(word, keywords[i]) == 0)
            return 1;
    }
    return 0;
}
%}

```

```
valid_id  [a-zA-Z_][a-zA-Z0-9_]*
invalid_id [0-9][a-zA-Z0-9_]*
```

```
%%
{valid_id} {
    if(isKeyword(yytext))
        printf("%s : Keyword (not identifier)\n", yytext);
    else
        printf("%s : Valid Identifier\n", yytext);
}

{invalid_id} {printf("%s : Invalid Identifier\n", yytext);}

[ \t\n]+ {}

. {printf("%s : Invalid Identifier\n", yytext);}
%%
```

```
int yywrap() {
    return 1;
}
```

```
int main() {
    printf("Enter tokens:\n");
    yylex();
    return 0;
}
```

14.

/* Write a LEX program to find and print all email IDs from a given text. */

```
%{
    #include <stdio.h>
}%

email [a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}

%%

{email} {printf("Email found: %s\n", yytext);}

.\n {}
%%

int yywrap() {
    return 1;
}
```

```
}
```

```
int main() {  
    printf("Inputs here:\n\n");  
    yylex();  
    return 0;  
}
```

15.

/* Write a LEX program to perform lexical analysis of a C program. */

```
%{  
    #include <stdio.h>  
    #include <string.h>  
}%
```

```
keyword    int|float|char|double|if|else|while|for|return|void|break|continue  
id          [a-zA-Z_][a-zA-Z0-9_]*  
number      [0-9]+(\.[0-9]+)?  
operator    "=="|"!="|"<="|">="|"++"|"--"|"+"|"-"|"*"|"/"|"="|"<"|">"  
delimiter   [;(),{}]
```

```
%%  
{keyword} {printf("Keyword: %s\n", yytext);}  
{id}      {printf("Identifier: %s\n", yytext);}  
{number}  {printf("Constant: %s\n", yytext);}  
{operator} {printf("Operator: %s\n", yytext);}  
{delimiter} {printf("Delimiter: %s\n", yytext);}  
[ \t\n]+ {}  
.        {printf("Unknown: %s\n", yytext);}  
%%
```

```
int yywrap(){  
    return 1;  
}
```

```
int main(){  
    printf("Enter the C code here:\n");  
    yylex();  
    return 0;  
}
```

16.

/* Write a LEX program to generate tokens for keywords, identifiers, constants, and operators. */

```
%{
```

```

#include <stdio.h>
%}

%%
int|float|char|double|if|else|while|for|return|void {printf("<KEYWORD, %s>\n", yytext);}
[a-zA-Z_][a-zA-Z0-9_]* {printf("<ID, %s>\n", yytext);}
[0-9]+(\.[0-9]+)? {printf("<CONST, %s>\n", yytext);}
"=="|"!="|"<="|">="|"+"| "-"|"*"|" "/"|"=" {printf("<OP, %s>\n", yytext);}
[ \t\n]+ {}
. {printf("<INVALID, %s>\n", yytext);}
%%

```

```

int yywrap(){
    return 1;
}

```

```

int main(){
    printf("Enter the C code here:\n");
    yylex();
    return 0;
}

```

17.

/* Write a LEX program to implement a symbol table. */

```

%{
#include <stdio.h>
#include <string.h>
char symtab[100][50];
int count = 0;
int exists(char *s){
    for(int i=0;i<count;i++)
        if(strcmp(symtab[i],s)==0) return 1;
    return 0;
}
%}

%%
int|float|char|double|if|else|while|for|return|void {}

[a-zA-Z_][a-zA-Z0-9_]* {
    if(!exists(yytext)){
        strcpy(symtab[count++], yytext);
        printf("Inserted into Symbol Table: %s\n", yytext);
    }
}

```



```

}

[ \t\n]+ ;
. ;
%%

int yywrap(){
    return 1;
}

int main(){
    printf("Enter the C code here:\n");
    yylex();
    printf("\nSymbol Table:\n");
    for(int i=0;i<count;i++)
        printf("%d. %s\n", i+1, symtab[i]);
    return 0;
}
18.
/* Write a LEX program to validate arithmetic expression */

%{
    #include <stdio.h>
    int valid = 1;
    int paren = 0;
}%

%%
[0-9]+        {}
[a-zA-Z_][a-zA-Z0-9_]* {}
"("          {paren++;}
")"          {paren--;if(paren<0) valid=0;}
[+ \- * /]    {}
[ \t\n]+      {}
.             {valid = 0;}
%%

int yywrap(){
    return 1;
}

int main(){
    printf("Enter expression:\n");
    yylex();

```

```

if(valid && paren==0)
    printf("Valid Arithmetic Expression\n");
else
    printf("Invalid Arithmetic Expression\n");
return 0;
}

```

Inputs and Outputs:

1.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q1.l; gcc lex.yy.c; ./a.exe

Input here: Hello World!

^Z

The number of characters in the given input is: 13.

```

* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q1.l; gcc lex.yy.c; ./a.exe

Input here: Hello World!
^Z
The number of characters in the given input is: 13.
* Terminal will be reused by tasks, press any key to close it.

```

2.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q2.l; gcc lex.yy.c; ./a.exe

Input here: Hello world, this is compiler assignment.

^Z

The number of words is: 6.

```

* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q2.l; gcc lex.yy.c; ./a.exe

Input here: Hello world, this is compiler assignment.
^Z
The number of words is: 6.
* Terminal will be reused by tasks, press any key to close it.

```

3.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q3.l; gcc lex.yy.c; ./a.exe

Input here: Hello

Bye

Compiler.

^Z

The number of lines is: 3.

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q3.1; gcc lex.yy.c; ./a.exe

Input here: Hello
Bye
Compiler.
^Z
The number of lines is: 3.
* Terminal will be reused by tasks, press any key to close it.
```

4.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q4.1; gcc lex.yy.c; ./a.exe

Input here: Hello World!

^Z

The number of vowels are: 3.

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q4.1; gcc lex.yy.c; ./a.exe

Input here: Hello World!
^Z
The number of vowels are: 3.
* Terminal will be reused by tasks, press any key to close it.
```

5.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q5.1; gcc lex.yy.c; ./a.exe

Input here: Hello world!

HELLO WORLD!

Bye.

BYE.

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q5.1; gcc lex.yy.c; ./a.exe

Input here: Hello world!
HELLO WORLD!
Bye.
BYE.
^Z
* Terminal will be reused by tasks, press any key to close it.
```

^Z

6.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q6.1; gcc lex.yy.c; ./a.exe

Input here: H3llo Fl4m3s!

^Z

The number of digits is: 4.

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q6.l; gcc lex.yy.c; ./a.exe
Input here: H3ll0 Fl4m3s!
^Z
The number of digits is: 4.
* Terminal will be reused by tasks, press any key to close it.
```

7.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q7.l; gcc lex.yy.c; ./a.exe

Write code here:

```
#include <stdio.h>
```

IDENTIFIER: include.

IDENTIFIER: stdio.

IDENTIFIER: h.

```
int main(){
```

KEYWORD: int.

IDENTIFIER: main.

```
    int a = 6 + 7;
```

KEYWORD: int.

IDENTIFIER: a.

```
    return 0;
```

KEYWORD: return.

```
}
```

^Z

The number of keywords is: 3.

The number of identifiers is: 5.

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q7.l; gcc lex.yy.c; ./a.exe

Write code here:
#include <stdio.h>
IDENTIFIER: include.
IDENTIFIER: stdio.
IDENTIFIER: h.
int main(){
KEYWORD: int.
IDENTIFIER: main.
    int a = 6 + 7;
KEYWORD: int.
IDENTIFIER: a.
    return 0;
KEYWORD: return.
}
^Z
The number of keywords is: 3.
The number of identifiers is: 5.
* Terminal will be reused by tasks, press any key to close it.
```

8.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q8.l; gcc lex.yy.c; ./a.exe

Input here:

11.11

Floating point value.

11

Integer.

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q8.l; gcc lex.yy.c; ./a.exe
Input here:
11.11
Floating point value.
11
Integer.
^Z
^Z_* Terminal will be reused by tasks, press any key to close it.
```

9.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q9.l; gcc lex.yy.c; ./a.exe

Inputs here:

a <=& b-- + ++c

^Z

Number of operands: 3.

Number of operators: 5.

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q9.l; gcc lex.yy.c; ./a.exe
Inputs here:
a <=& b-- + ++c
^Z
Number of operands: 3.
Number of operators: 5.
_* Terminal will be reused by tasks, press any key to close it.
```

10.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q10.l; gcc lex.yy.c; ./a.exe

Enter C Code:

#include <stdio.h>

//Single line comment.

/* Multi

line

comment.

*/

int main(){

int a = 6 + 7;

return 0;

}

^Z

The C code without any comments is:

```
#include <stdio.h>
```

```
int main(){  
    int a = 6 + 7;  
    return 0;
```

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q10.l; gcc lex.yy.c; ./a.exe  
  
Enter C Code:  
#include <stdio.h>  
//Single line comment.  
/* Multi  
line  
comment.  
*/  
int main(){  
    int a = 6 + 7;  
    return 0;  
}  
^Z  
The C code without any comments is:  
  
#include <stdio.h>  
  
  
  
int main(){  
    int a = 6 + 7;  
    return 0;  
}  
* Terminal will be reused by tasks, press any key to close it.
```

11.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q11.l; gcc lex.yy.c; ./a.exe

Inputs here:

<

RELATIONAL OPERATOR: <.

<=

RELATIONAL OPERATOR: <=.

&&

LOGICAL OPERATOR: &&.

&

||
LOGICAL OPERATOR: ||.

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q11.l; gcc lex.yy.c; ./a.exe

Inputs here:
<
RELATIONAL OPERATOR: <.
<=
RELATIONAL OPERATOR: <=.
&&
LOGICAL OPERATOR: &&.
&
||
LOGICAL OPERATOR: ||.
^Z
* Terminal will be reused by tasks, press any key to close it.
```

12.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q12.l; gcc lex.yy.c; ./a.exe

Input here: Hello World!

^Z

Number of uppercase letters is: 2.

Number of lowercase letters is: 8.

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q12.l; gcc lex.yy.c; ./a.exe

Input here: Hello World!
^Z
Number of uppercase letters is: 2.
Number of lowercase letters is: 8.
* Terminal will be reused by tasks, press any key to close it.
```

13.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q13.l; gcc lex.yy.c; ./a.exe

Enter tokens:

int sit

int : Keyword (not identifier)

sit : Valid Identifier

9ab-*bc

9ab : Invalid Identifier

- : Invalid Identifier

* : Invalid Identifier

bc : Valid Identifier

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q13.1; gcc lex.yy.c; ./a.exe

Enter tokens:
int sit
int : Keyword (not identifier)
sit : Valid Identifier
9ab-*bc
9ab : Invalid Identifier
- : Invalid Identifier
* : Invalid Identifier
bc : Valid Identifier
^Z^Z
```

14.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q14.1; gcc lex.yy.c; ./a.exe

Inputs here:

```
ab
vishwajith.s2023@vitstudent.ac.in
Email found: vishwajith.s2023@vitstudent.ac.in
something@something.something
Email found: something@something.something
```

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q14.1; gcc lex.yy.c; ./a.exe

Inputs here:

ab
vishwajith.s2023@vitstudent.ac.in
Email found: vishwajith.s2023@vitstudent.ac.in
something@something.something
Email found: something@something.something
^Z
* Terminal will be reused by tasks, press any key to close it.
```

15.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q15.1; gcc lex.yy.c; ./a.exe

Enter the C code here:

```
#include <stdio.h>
```

Unknown: #

Identifier: include

Operator: <

Identifier: stdio

Unknown: .

Identifier: h

Operator: >

```
int main(){
```

Keyword: int

Identifier: main
Delimiter: (
Delimiter:)
Delimiter: {
 int a = 6+7;
Keyword: int
Identifier: a
Operator: =
Constant: 6
Operator: +
Constant: 7
Delimiter: ;
 return 0;
Keyword: return
Constant: 0
Delimiter: ;
}
Delimiter: }

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\week-7"; flex Q15.1; gcc lex.yy.c; ./a.exe

Enter the C code here:
#include <stdio.h>
Unknown: #
Identifier: include
Operator: <
Identifier: stdio
Unknown: .
Identifier: h
Operator: >
int main(){
Keyword: int
Identifier: main
Delimiter: (
Delimiter: )
Delimiter: {
    int a = 6+7;
Keyword: int
Identifier: a
Operator: =
Constant: 6
Operator: +
Constant: 7
Delimiter: ;
    return 0;
Keyword: return
Constant: 0
Delimiter: ;
}
Delimiter: }
^Z
* Terminal will be reused by tasks, press any key to close it.
```

16.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q16.l; gcc lex.yy.c; ./a.exe

Enter the C code here:

```
int main(){
<KEYWORD, int>
<ID, main>
<INVALID, (>
<INVALID, )>
<INVALID, {>
    return 0;
<KEYWORD, return>
<CONST, 0>
<INVALID, ;>
}
<INVALID, }>
```

17.

Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q17.l; gcc lex.yy.c; ./a.exe

Enter the C code here:

```
int main(){
Inserted into Symbol Table: main
    return 0;
}
^Z
```

Symbol Table:

1. main

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q17.l; gcc lex.yy.c; ./a.exe

Enter the C code here:
int main(){
Inserted into Symbol Table: main
    return 0;
}
^Z

Symbol Table:
1. main
* Terminal will be reused by tasks, press any key to close it.
```

18.

```
Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q18.l; gcc
lex.yy.c; ./a.exe
```

Enter expression:

a + (b - c) * d - ((e + f) / g)

^Z

Valid Arithmetic Expression

```
* Executing task: cd "C:\Users\vishw\Coding\Compiler-Lab\Week-7"; flex Q18.l; gcc lex.yy.c; ./a.exe

Enter expression:
a + (b - c) * d - ((e + f) / g)
^Z
Valid Arithmetic Expression
* Terminal will be reused by tasks, press any key to close it.
```

Result:

The LEX programs written for the given questions were successfully compiled and run.

Compiler Design Lab

Date: 13-2-2026
Assignment No.: 7

Name: S Vishwajith
Register No.: 23BCE1145

Aim:

To implement the SLR parser using C++.

Code:

```
#include <iostream>
#include <iomanip>
#include <vector>
#include <set>
#include <map>
using namespace std;

struct prod
{
    int lhsIdx;
    vector<pair<int, int>> rhsIdx; // nt/t, nt/tIdx - char
};

struct stackItem
{
    int type; // 0-state, 1-NT, 2-T
    int state;
    int ntIdx;
    int tIdx;
};

char startSym;
int startSymIdx;
set<char> terms, nterms;
vector<prod> prods;
vector<pair<int, int>> litems; // prodIdx, .pos - lr0 item
vector<vector<int>> lstates; // stateNo - vector(litemsIdx)
vector<vector<int>> closures; // ntIdx - vector(litemsIdx)
vector<int> cproc;
vector<int> fiproc, foproc;
vector<set<char>> first, follow; // ntIdx - set(first) and set(follow)
```

```

map<pair<int, pair<int, int>>, int> gotos; // stateNo, (nt/t nt/tldx) -> stateNo
vector<vector<string>> table;           // stateNo, t-ntldx - Action
string input;
vector<stackItem> stack;

int findState(vector<int> v)
{
    set<int> state(v.begin(), v.end());
    for (int i = 0; i < lstates.size(); i++)
    {
        set<int> cstate(lstates[i].begin(), lstates[i].end());
        if (state == cstate)
            return i;
    }
    return -1;
}

int findLitem(pair<int, int> litem, vector<pair<int, int>> litems)
{
    for (int i = 0; i < litems.size(); i++)
    {
        if (litem == litems[i])
            return i;
    }
    return -1;
}

int findInt(int i, vector<int> v)
{
    for (int j = 0; j < v.size(); j++)
    {
        if (i == v[j])
            return j;
    }
    return -1;
}

set<char> fillFirst(char t)
{
    int ntldx = distance(nterms.begin(), nterms.find(t));
    if (fiproc[ntldx])
        return first[ntldx];
    fiproc[ntldx] = 1;
    for (prod p : prods)

```

```

{
    if (p.lhsIdx == ntIdx)
    {
        int fnt = p.rhsIdx[0].first;
        int fldx = p.rhsIdx[0].second;
        if (fnt)
            first[ntIdx].insert(*next(terms.begin(), fldx));
        else
        {
            char tgt = *next(nterms.begin(), fldx);
            for (char c : fillFirst(tgt))
                first[ntIdx].insert(c);
        }
    }
}
return first[ntIdx];
}

set<char> fillFollow(char t)
{
    int ntIdx = distance(nterms.begin(), nterms.find(t));
    if (foproc[ntIdx])
        return follow[ntIdx];
    foproc[ntIdx] = 1;
    for (prod p : prods)
    {
        for (int i = 0; i < p.rhsIdx.size(); i++)
        {
            int fnt = p.rhsIdx[i].first;
            int fntIdx = p.rhsIdx[i].second;
            if (fnt == 0 && fntIdx == ntIdx)
            {
                if (i == p.rhsIdx.size() - 1)
                {
                    char tgt = *next(nterms.begin(), p.lhsIdx);
                    for (char c : fillFollow(tgt))
                        follow[ntIdx].insert(c);
                }
            }
            else
            {
                int fn = p.rhsIdx[i + 1].first;
                int fnIdx = p.rhsIdx[i + 1].second;
                if (fn)
                    follow[ntIdx].insert(*next(terms.begin(), fnIdx));
            }
        }
    }
}

```

```

        else
        {
            char tgt = *next(nterms.begin(), fnIdx);
            for (char c : fillFirst(tgt))
                follow[ntIdx].insert(c);
        }
    }
}

return follow[ntIdx];
}

vector<int> findClosure(int nt)
{
    if (cproc[nt])
        return closures[nt];
    cproc[nt] = 1;
    vector<int> litems;
    for (int i = 0; i < prods.size(); i++)
    {
        if (prods[i].lhsIdx == nt)
        {
            litems.push_back(findLitem(pair<int, int>(i, 0), ::litems));
            if (prods[i].rhsIdx[0].first == 0 &&
                prods[i].rhsIdx[0].second != nt)
            {
                vector<int> vc = findClosure(prods[i].rhsIdx[0].second);
                for (int i : vc)
                {
                    if (findInt(i, litems) == -1)
                        litems.push_back(i);
                }
            }
        }
    }
    closures[nt] = litems;
    return litems;
}

void constructClosures()
{
    for (int i = 0; i < prods.size(); i++)

```

```

        litems.push_back(pair<int, int>(i, 0));
cproc = vector<int>(nterms.size(), 0);
closures = vector<vector<int>>(nterms.size());
for (int i = 0; i < nterms.size(); i++)
    findClosure(i);
}

void printProd(prod pro, int i)
{
    char l = *next(nterms.begin(), pro.lhsIdx);
    if (l == 'Z')
        cout << startSym << "" << "->";
    else
        cout << l << "->";
    int x = 0;
    for (pair<int, int> ridx : pro.rhsIdx)
    {
        char c;
        if (ridx.first == 0)
            c = *next(nterms.begin(), ridx.second);
        else
            c = *next(terms.begin(), ridx.second);
        if (x++ == i)
            cout << ".";
        cout << c;
    }

    if (i == pro.rhsIdx.size())
        cout << ".";
    cout << endl;
}

void findGoto(int lstateNo, char c)
{
    int nt = 1;
    if (nterms.count(c))
        nt = 0;
    int ntIdx;
    if (nt == 0)
        ntIdx = distance(nterms.begin(), nterms.find(c));
    else
        ntIdx = distance(terms.begin(), terms.find(c));
    vector<int> litems;

```



```

int found = 0;
for (int x : lstates[lstateNo])
{
    pair<int, int> p = litems[x];
    prod pr = prods[p.first];
    int pos = p.second;
    if (pr.rhsldx.size() > pos)
    {
        if (pair<int, int>(nt, ntldx) == pr.rhsldx[pos])
        {
            found = 1;
            pair<int, int> newitem(p.first, pos + 1);
            int newldx = findLitem(newitem, litems);
            if (newldx == -1)
            {
                litems.push_back(newitem);
                litems.push_back(litems.size() - 1);
            }
            else
                litems.push_back(newldx);
            if (pos + 1 < pr.rhsldx.size())
            {
                pair<int, int> clositem = pr.rhsldx[pos + 1];
                if (clositem.first == 0)
                {
                    vector<int> itemclos = closures[clositem.second];
                    for (int i : itemclos)
                    {
                        if (findInt(i, litems) == -1)
                            litems.push_back(i);
                    }
                }
            }
        }
    }
}

pair<int, int> nti(nt, ntldx);
pair<int, pair<int, int>> gt(lstateNo, nti);
int gtstatenum = -1;
if (!found)
{
    pair<pair<int, pair<int, int>>, int> val(gt, gtstatenum);
    gotos.insert(val);
}

```

```

else if (findState(lritems) == -1)
{
    lstates.push_back(lritems);
    gtstatenum = lstates.size() - 1;
}
else
    gtstatenum = findState(lritems);
pair<pair<int, pair<int, int>>, int> val(gt, gtstatenum);
gotos.insert(val);
}

void printGotos()
{
    int x = 0;
    int s = nterms.size() + terms.size() - 1;
    for (pair<pair<int, pair<int, int>>, int> p : gotos)
    {
        int nt = p.first.second.first;
        int ntldx = p.first.second.second;
        char ntc;
        if (nt)
            ntc = *next(terms.begin(), ntldx);
        else
            ntc = *next(nterms.begin(), ntldx);
        int gst = p.second;
        if (gst != -1)
            cout << "goto(" << p.first.first << ", " << ntc << "): " << gst << " | ";
        else
            cout << "goto(" << p.first.first << ", " << ntc << "): - | ";
        x = (x + 1) % s;
        if (x == 0)
            cout << endl;
    }
}

void printClosures()
{
    for (int i = 0; i < closures.size(); i++)
    {
        cout << *next(nterms.begin(), i) << ": " << endl;
        for (int p : closures[i])
        {
            pair<int, int> pr = *next(lititems.begin(), p);
            prod pro = prods[pr.first];

```

```

        printProd(pro, pr.second);
    }
    cout << endl;
}

void printStates()
{
    for (int j = 0; j < lstates.size(); j++)
    {
        cout << "I" << j << endl;
        for (int p : lstates[j])
        {
            pair<int, int> pr = *next(litems.begin(), p);
            prod pro = prods[pr.first];
            printProd(pro, pr.second);
        }
        cout << endl;
    }
}

void printFollows()
{
    for (int i = 0; i < follow.size(); i++)
    {
        char c = *next(nterms.begin(), i);
        cout << c << ": {";
        for (char x : follow[i])
            cout << x << " ";
        cout << "}" << endl;
    }
    cout << endl;
}

void fillTable()
{
    for (pair<pair<int, pair<int, int>>, int> p : gotos)
    {
        int gst = p.second;
        int sn = p.first.first;
        int nt = p.first.second.first;
        int ntldx = p.first.second.second;
        if (gst == -1)
            continue;
    }
}

```

```

    if (nt)
        table[sn][ntldx] = "s" + to_string(gst);
    else if (startSymldx != ntldx)
        table[sn][ntldx + terms.size()] = to_string(gst);
}
for (int i = 0; i < lstates.size(); i++)
{
    for (int litem : lstates[i])
    {
        int pos = litems[litem].second;
        int prodldx = litems[litem].first;
        if (pos == prods[prodldx].rhsldx.size())
        {
            for (char c : terms)
            {
                int cldx = distance(terms.begin(), terms.find(c));
                if (follow[prods[prodldx].lhsldx].count(c))
                {
                    string fstring;
                    if (prodldx == 0)
                        fstring = "ACC";
                    else
                        fstring = "r" + to_string(prodldx);
                    if (table[i][cldx] == "-")
                        table[i][cldx] = fstring;
                }
            }
        }
    }
}
}
}

```

```

void printTable()
{
    cout << "SLR Parsing Table: " << endl;
    int nnt = nterms.size() - 1;
    int ntt = terms.size();
    cout << setw(5) << "STATE"
        << setw(4) << "|"
        << setw(ntt * 5) << "ACTION"
        << setw(ntt * 5) << "|"
        << setw(nnt * 2) << "GOTO"
        << setw(nnt * 2) << "|" << endl;
    cout << setw(5) << "- "

```

```

        << setw(4) << "|";
    for (char c : terms)
        cout << setw(5) << c << setw(5) << "|";
    for (char c : nterms)
    {
        if (c != 'Z')
            cout << setw(2) << c << setw(2) << "|";
    }
    cout << endl;
    for (int i = 0; i < table.size(); i++)
    {
        cout << setw(5) << i << setw(4) << "|";
        int j = 0;
        for (; j < terms.size(); j++)
            cout << setw(5) << table[i][j] << setw(5) << "|";
        for (; j < table[i].size() - 1; j++)
            cout << setw(2) << table[i][j] << setw(2) << "|";
        cout << endl;
    }
}

```

```

void printStack()
{
    string out;
    for (stackItem s : stack)
    {
        if (s.type == 0)
            out += to_string(s.state);
        else if (s.type == 1)
            out += *next(nterms.begin(), s.ntldx);
        else if (s.type == 2)
            out += *next(terms.begin(), s.tldx);
    }
    cout << setw(10) << out << setw(10) << "|";
}

```

```

void parse(string in)
{
    in = in + '$';
    int i = 0;
    int j = 0;
    int acc = 0;
    stackItem init;
    init.type = 0;
}

```

```

init.state = 0;
stack.push_back(init);
cout << setw(10) << "STACK"
    << setw(10) << "|"
    << setw(10) << "INPUT"
    << setw(10) << "|"
    << setw(20) << "ACTION"
    << setw(20) << "|" << endl;
while (i < in.size() && acc == 0)
{
    printStack();
    cout << setw(10) << in.substr(i) << setw(10) << "|";
    int tldx = distance(terms.begin(), terms.find(in[i]));
    string a = table[stack[j].state][tldx];
    if (a[0] == 's')
    {
        j += 2;
        stackItem newItem;
        newItem.type = 2;
        newItem.tldx = distance(terms.begin(), terms.find(in[i++]));
        stack.push_back(newItem);
        newItem.type = 0;
        newItem.state = stoi(a.substr(1));
        newItem.tldx = -1;
        stack.push_back(newItem);
        cout << setw(20) << "SHIFT " + to_string(newItem.state)
            << setw(20) << "|" << endl;
    }
    else if (a[0] == 'A')
    {
        cout << setw(20) << "ACCEPT"
            << setw(20) << "|" << endl;
        exit(0);
    }
    else if (a[0] == 'r')
    {
        int prodIdx = stoi(a.substr(1));
        prod rprod = prods[prodIdx];
        vector<pair<int, int>> rldx;
        int ntop = j - 2 * rprod.rhsIdx.size() + 1;
        if (ntop < 0)
        {
            cout << setw(20) << "ERROR"
                << setw(20) << "|" << endl;
        }
    }
}

```

```

    exit(0);
}
for (int x = ntop; x <= j; x += 2)
{
    stackItem s1 = stack[x];
    rldx.push_back(
        pair<int, int>(
            s1.type - 1,
            s1.type == 1 ? s1.ntldx : s1.tldx));
}
while (j >= ntop)
{
    j--;
    stack.pop_back();
}
if (rldx == rprod.rhsldx)
{
    stackItem s2;
    s2.type = 1;
    s2.ntldx = rprod.lhsldx;
    int gt = stoi(
        table[stack[j].state][terms.size() + s2.ntldx]);
    stackItem s3;
    s3.type = 0;
    s3.state = gt;
    stack.push_back(s2);
    stack.push_back(s3);
    j += 2;
    char l = *next(nterms.begin(), rprod.lhsldx);
    string nout = string(1, l) + "->";
    for (pair<int, int> ridx : rprod.rhsldx)
    {
        char c;
        if (ridx.first == 0)
            c = *next(nterms.begin(), ridx.second);
        else
            c = *next(terms.begin(), ridx.second);
        nout += c;
    }
    cout << setw(20) << "REDUCE " + nout
        << setw(20) << "|" << endl;
}
else
{

```

```

        cout << setw(30) << "REDUCE FAILED"
            << setw(10) << "|" << endl;
        cout << "Stack RHS: ";
        for (pair<int, int> p : rldx)
        {
            char c;
            if (p.first == 0)
                c = *next(nterms.begin(), p.second);
            else
                c = *next(terms.begin(), p.second);
            cout << c << " ";
        }
        cout << endl
            << "Target RHS: ";
        for (pair<int, int> p : rprod.rhsldx)
        {
            char c;
            if (p.first == 0)
                c = *next(nterms.begin(), p.second);
            else
                c = *next(terms.begin(), p.second);
            cout << c << " ";
        }
        exit(0);
    }
}
else
{
    cout << setw(20) << "ERROR"
        << setw(20) << "|" << endl;
    exit(0);
}
}
}

int main()
{
    int stlr = 0;
    string in, t;
    vector<pair<int, int>> v;
    vector<prod> tempProds;
    prod p;
    cout << "Enter terminals seperated by comma: ";
    cin >> in;

```



```

for (int i = 0; i < in.size(); i += 2)
    terms.insert(in[i]);
cout << "Enter non-terminals seperated by comma: ";
cin >> in;
for (int i = 0; i < in.size(); i += 2)
    nterms.insert(in[i]);
terms.insert('$');
cout << "Enter starting symbol: ";
cin >> startSym;
cout << "Enter productions seperated by |: ";
nterms.insert('Z');
startSymIdx = distance(nterms.begin(), nterms.find('Z'));
cin >> in;
for (int i = 0; i < in.size(); i++)
{
    char c = in[i];
    if (c == '|')
    {
        p.rhsIdx = v;
        tempProds.push_back(p);
        p.rhsIdx.clear();
        v.clear();
        stlr = 0;
    }
    else if (c == '-')
    {
        stlr = 1;
        i++;
    }
    else if (stlr == 0)
        p.lhsIdx = distance(nterms.begin(), nterms.find(c));
    else if (terms.count(c))
        v.push_back(
            pair<int, int>(
                1,
                distance(terms.begin(), terms.find(c))));
    else if (nterms.count(c))
        v.push_back(
            pair<int, int>(
                0,
                distance(nterms.begin(), nterms.find(c))));
    else
        cout << "ERROR";
}

```

```

cout << "Enter input string: ";
cin >> input;
p.rhsIdx = v;
tempProds.push_back(p);
p.rhsIdx.clear();
v.clear();
p.lhsIdx = distance(nterms.begin(), nterms.find('Z'));
v.push_back(
    pair<int, int>(
        0,
        distance(nterms.begin(), nterms.find(startSym))));
p.rhsIdx = v;
prods.push_back(p);
for (prod pr : tempProds)
    prods.push_back(pr);
cout << endl
    << "Augmented productions: " << endl;
for (int i = 0; i < prods.size(); i++)
{
    cout << i << ". ";
    printProd(prods[i], -1);
}
fiproc = vector<int>(nterms.size(), 0);
foproc = vector<int>(nterms.size(), 0);
first = vector<set<char>>(nterms.size());
follow = vector<set<char>>(nterms.size());
follow[startSymIdx].insert('$');
for (char c : nterms)
    fillFollow(c);
constructClosures();
lstates.push_back(findClosure(startSymIdx));
for (int i = 0; i < lstates.size(); i++)
{
    for (char c : terms)
    {
        if (c != '$')
            findGoto(i, c);
    }
    for (char c : nterms)
    {
        if (c != 'Z')
            findGoto(i, c);
    }
}
}

```

```

cout << endl
    << "States: " << endl;
printStats();
cout << "Gotos: " << endl;
printGotos();
cout << endl
    << endl;
cout << "Follow(): " << endl;
printFollows();
table = vector<vector<string>>(
    lstates.size(),
    vector<string>(nterms.size() + terms.size(), "-"));
fillTable();
printTable();
cout << endl
    << "Parsing Input: " << endl;
parse(input);
}

// S->L=R|S->R|L->*R|L->id|R->L

```

Input and Output:

Input Grammar: (NT = {i, d, =, *}, T = {S, L, R}, P={ S->L=R | S->R | L->*R | L->id | R->L}, S = {S})

Input String: *d=id

```

PS C:\Users\vishw> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-8\" ; if ($?) { g++ SLR_Parser.cpp -o SLR_Parser } ; if ($?) { .\SLR_Parser }
Enter terminals seperated by comma: =,*,i,d
Enter non-terminals seperated by comma: S,L,R
Enter starting symbol: S
Enter productions seperated by |: S->L=R|S->R|L->*R|L->id|R->L
Enter input string: id=id

Augmented productions:
0. S'->S
1. S->L=R
2. S->R
3. L->*R
4. L->id
5. R->L

States:
I0
S'->.S
S->L.=R
L->.*R
L->.id
S->R
R->L

I1
L->*R
R->L
L->.*R
L->.id

I2
L->i.d

I3
S->L.=R
R->L.

I4
S->R.

I5
S'->S.

I6
R->L.

I7
L->*R.

I8
L->id.

I9
S->L.=R
R->L.
L->.*R
L->.id

I10
S->L=R.

Gotos:
goto(0,L): 3 | goto(0,R): 4 | goto(0,S): 5 | goto(0,*) : 1 | goto(0,=) : - | goto(0,d) : - | goto(0,i): 2 | goto(1,L): 6 |
goto(1,R): 7 | goto(1,S) : - | goto(1,*) : 1 | goto(1,=) : - | goto(1,d) : - | goto(1,i): 2 | goto(2,L): - | goto(2,R): - |
goto(2,S) : - | goto(2,*) : - | goto(2,=) : - | goto(2,d): 8 | goto(2,i) : - | goto(3,L): - | goto(3,R): - | goto(3,S): - |
goto(3,*) : - | goto(3,=) : 9 | goto(3,d) : - | goto(3,i) : - | goto(4,L): - | goto(4,R): - | goto(4,S): - | goto(4,*) : - |
goto(4,=) : - | goto(4,d) : - | goto(4,i) : - | goto(5,L): - | goto(5,R): - | goto(5,S): - | goto(5,*) : - | goto(5,=) : - |
goto(5,d) : - | goto(5,i) : - | goto(6,L): - | goto(6,R): - | goto(6,S): - | goto(6,*) : - | goto(6,=) : - | goto(6,d) : - |
goto(6,i) : - | goto(7,L): - | goto(7,R): - | goto(7,S): - | goto(7,*) : - | goto(7,=) : - | goto(7,d) : - | goto(7,i) : - |
goto(8,L): - | goto(8,R): - | goto(8,S): - | goto(8,*) : - | goto(8,=) : - | goto(8,d) : - | goto(8,i) : - | goto(9,L): 6 |
goto(9,R): 10 | goto(9,S) : - | goto(9,*) : 1 | goto(9,=) : - | goto(9,d) : - | goto(9,i): 2 | goto(10,L): - | goto(10,R): - |
goto(10,S): - | goto(10,*) : - | goto(10,=) : - | goto(10,d) : - | goto(10,i) : - |

Follow():
L: {$ = }
R: {$ = }
S: {$ }
Z: {$ }

```

SLR Parsing Table:

STATE			ACTION				GOTO		
							L	R	S
-	\$	*	=	d	i				
0	-	s1	-	-	s2		3	4	5
1	-	s1	-	-	s2		6	7	-
2	-	-	-	s8	-		-	-	-
3	r5	-	s9	-	-		-	-	-
4	r2	-	-	-	-		-	-	-
5	ACC	-	-	-	-		-	-	-
6	r5	-	r5	-	-		-	-	-
7	r3	-	r3	-	-		-	-	-
8	r4	-	r4	-	-		-	-	-
9	-	s1	-	-	s2		6	10	-
10	r1	-	-	-	-		-	-	-

Parsing Input:

STACK	INPUT	ACTION
0	id=id\$	SHIFT 2
0i2	d=id\$	SHIFT 8
0i2d8	=id\$	REDUCE L->id
0L3	=id\$	SHIFT 9
0L3=9	id\$	SHIFT 2
0L3=9i2	d\$	SHIFT 8
0L3=9i2d8	\$	REDUCE L->id
0L3=9L6	\$	REDUCE R->L
0L3=9R10	\$	REDUCE S->L=R
0S5	\$	ACCEPT

PS C:\Users\vishw\Coding\Compiler-Lab\Week-8> |

Result:

The SLR parser has been successfully written in C++ and executes as expected.

Compiler Design Lab

Date: 27-2-2026
Assignment No.: 8

Name: S Vishwajith
Register No.: 23BCE1145

Aim:

To implement a CLR parser using C++.

Code:

```
#include <iostream>
#include <vector>
#include <string>
#include <map>
#include <set>
#include <stack>
#include <algorithm>
#include <iomanip>
using namespace std;

vector<pair<string, vector<string>>> G = {
    {"S", {"S"}},
    {"S", {"C", "C"}},
    {"C", {"c", "C"}},
    {"C", {"d"}},
};

const string INPUT = "ccdd";
set<string> NT, TM;
string AUG;

bool isNT(const string &s) { return NT.count(s) > 0; }

void deriveSymbols()
{
    AUG = G[0].first;
    for (auto &[l, r] : G)
        NT.insert(l);
    for (auto &[l, r] : G)
        for (auto &s : r)
```

```

        if (!NT.count(s))
            TM.insert(s);
    TM.insert("$");
}

map<string, set<string>> FIRST;

set<string> firstSeq(const vector<string> &v, int from = 0)
{
    set<string> res;
    if (from >= (int)v.size())
    {
        res.insert("eps");
        return res;
    }
    for (int i = from; i < (int)v.size(); i++)
    {
        const string &s = v[i];
        set<string> fi = TM.count(s) ? set<string>{s} : FIRST[s];
        bool hasEps = fi.count("eps");
        for (auto &x : fi)
            if (x != "eps")
                res.insert(x);
        if (!hasEps)
            return res;
    }
    res.insert("eps");
    return res;
}

void computeFirst()
{
    for (auto &n : NT)
        FIRST[n] = {};
    bool changed = true;
    while (changed)
    {
        changed = false;
        for (auto &[l, r] : G)
            for (auto &x : firstSeq(r))
                if (!FIRST[l].count(x))
                {
                    FIRST[l].insert(x);
                    changed = true;
                }
    }
}

```

```

    }
}
}

```

```

struct Item
{
    int p, d;
    string la;
    bool operator<(const Item &o) const { return tie(p, d, la) < tie(o.p, o.d, o.la); }
    bool operator==(const Item &o) const { return p == o.p && d == o.d && la == o.la; }
};
using ISet = set<Item>;

```

```

string itemStr(const Item &it)
{
    auto &[lhs, rhs] = G[it.p];
    string s = lhs + "->";
    for (int i = 0; i <= (int)rhs.size(); i++)
    {
        if (i == it.d)
            s += ".";
        if (i < (int)rhs.size())
            s += " " + rhs[i];
    }
    s += " [" + it.la + "]";
    return s;
}

```

```

ISet closure1(ISet s)
{
    vector<Item> wl(s.begin(), s.end());
    while (!wl.empty())
    {
        Item it = wl.back();
        wl.pop_back();
        auto &rhs = G[it.p].second;
        if (it.d >= (int)rhs.size())
            continue;
        const string &B = rhs[it.d];
        if (!isNT(B))
            continue;
        vector<string> beta(rhs.begin() + it.d + 1, rhs.end());
        beta.push_back(it.la);
        for (auto &b : firstSeq(beta))

```



```

    {
        if (b == "eps")
            continue;
        for (int i = 0; i < (int)G.size(); i++)
        {
            if (G[i].first != B)
                continue;
            Item ni{i, 0, b};
            if (!s.count(ni))
            {
                s.insert(ni);
                wl.push_back(ni);
            }
        }
    }
}
return s;
}

/Set gotoSt(const /Set &s, const string &sym)
{
    /Set kernel;
    for (auto &it : s)
    {
        auto &rhs = G[it.p].second;
        if (it.d < (int)rhs.size() && rhs[it.d] == sym)
            kernel.insert({it.p, it.d + 1, it.la});
    }
    return kernel.empty() ? kernel : closure1(kernel);
}

```

```

vector</Set> STATES;
map<pair<int, string>, int> TRANS;

```

```

void buildCollection()
{
    STATES.push_back(closure1({{0, 0, "$"}}));
    bool changed = true;
    while (changed)
    {
        changed = false;
        int n = (int)STATES.size();
        for (int i = 0; i < n; i++)
        {

```

```

    set<string> syms;
    for (auto &it : STATES[i])
    {
        auto &rhs = G[it.p].second;
        if (it.d < (int)rhs.size())
            syms.insert(rhs[it.d]);
    }
    for (auto &sym : syms)
    {
        /Set g = gotoSt(STATES[i], sym);
        if (g.empty())
            continue;
        int idx = -1;
        for (int j = 0; j < (int)STATES.size(); j++)
            if (STATES[j] == g)
            {
                idx = j;
                break;
            }
        if (idx == -1)
        {
            idx = (int)STATES.size();
            STATES.push_back(g);
            changed = true;
        }
        TRANS[{i, sym}] = idx;
    }
}
}
}

```

```

map<int, map<string, string>> ACT;
map<int, map<string, int>> GOT;
vector<string> conflicts;

```

```

void buildTables()
{
    for (int i = 0; i < (int)STATES.size(); i++)
    {
        for (auto &it : STATES[i])
        {
            auto &[lhs, rhs] = G[it.p];
            if (it.d < (int)rhs.size())
            {

```

```

const string &sym = rhs[it.d];
if (TM.count(sym) && sym != "$")
{
    if (TRANS.count({i, sym}))
    {
        string act = "s" + to_string(TRANS[{i, sym}]);
        if (ACT[i].count(sym) && ACT[i][sym] != act)
            conflicts.push_back("State " + to_string(i) + " on " + sym +
                                ": " + ACT[i][sym] + " vs " + act);
        ACT[i][sym] = act;
    }
}
else if (isNT(sym))
{
    if (TRANS.count({i, sym}))
        GOT[i][sym] = TRANS[{i, sym}];
}
}
else
{
    if (lhs == AUG && it.la == "$")
    {
        ACT[i]["$"] = "acc";
    }
    else
    {
        string act = "r" + to_string(it.p);
        if (ACT[i].count(it.la) && ACT[i][it.la] != act)
            conflicts.push_back("State " + to_string(i) + " on " + it.la +
                                ": " + ACT[i][it.la] + " vs " + act);
        ACT[i][it.la] = act;
    }
}
}
}
}

```

```

vector<int> colWidths(const vector<string> &hdr, const vector<vector<string>> &rows)
{
    vector<int> w(hdr.size(), 0);
    for (int i = 0; i < (int)hdr.size(); i++)
        w[i] = (int)hdr[i].size();
    for (auto &r : rows)
        for (int i = 0; i < (int)r.size() && i < (int)w.size(); i++)

```

```

        w[i] = max(w[i], (int)r[i].size());
    return w;
}

void hline(const vector<int> &w, char fc = '-')
{
    cout << "+";
    for (auto x : w)
        cout << string(x + 2, fc) << "+";
    cout << "\n";
}

void printRow(const vector<int> &w, const vector<string> &cells)
{
    cout << "|";
    for (int i = 0; i < (int)cells.size(); i++)
    {
        string c = i < (int)cells.size() ? cells[i] : "";
        cout << " " << left << setw(w[i]) << c << " |";
    }
    cout << "\n";
}

void printTable(const vector<string> &hdr, const vector<vector<string>> &rows)
{
    auto w = colWidths(hdr, rows);
    hline(w, '=');
    printRow(w, hdr);
    hline(w, '=');
    for (auto &r : rows)
    {
        printRow(w, r);
        hline(w, '-');
    }
}

void sectionBanner(const string &title)
{
    int n = (int)title.size() + 4;
    cout << "\n"
        << string(n, '=') << "\n";
    cout << "| " << title << " |\n";
    cout << string(n, '=') << "\n\n";
}

```

```

void printGrammar()
{
    sectionBanner("GRAMMAR (Augmented)");
    for (int i = 0; i < (int)G.size(); i++)
    {
        cout << " (" << i << ") " << G[i].first << " -> ";
        if (G[i].second.empty())
            cout << "eps";
        else
            for (auto &s : G[i].second)
                cout << s << " ";
        cout << "\n";
    }
}

void printFirstSets()
{
    sectionBanner("FIRST SETS");
    vector<string> ntVec;
    for (auto &n : NT)
        if (n != AUG)
            ntVec.push_back(n);
    sort(ntVec.begin(), ntVec.end());
    for (auto &n : ntVec)
    {
        cout << " FIRST( " << n << " ) = { ";
        bool first = true;
        for (auto &x : FIRST[n])
        {
            if (!first)
                cout << ", ";
            cout << x;
            first = false;
        }
        cout << " }\n";
    }
}

void printStates()
{
    sectionBanner("CANONICAL LR(1) ITEM SETS");
    for (int i = 0; i < (int)STATES.size(); i++)
    {

```

```

    string hdr = " State I" + to_string(i) + " ";
    cout << "\n"
        << hdr << string(max(0, 44 - (int)hdr.size()), '-') << "\n";
    for (auto &it : STATES[i])
        cout << "    " << itemStr(it) << "\n";
    bool any = false;
    for (auto &[key, val] : TRANS)
        if (key.first == i)
        {
            any = true;
            break;
        }
    if (any)
    {
        cout << "    " << string(36, '-') << "\n";
        for (auto &[key, val] : TRANS)
            if (key.first == i)
                cout << "    goto( " << key.second << " ) => I" << val << "\n";
    }
}
}

```

```

void printParsingTable()
{
    sectionBanner("CLR(1) PARSING TABLE");

    vector<string> termCols, ntCols;
    for (auto &t : TM)
        if (t != "$")
            termCols.push_back(t);
    sort(termCols.begin(), termCols.end());
    termCols.push_back("$");

    for (auto &n : NT)
        if (n != AUG)
            ntCols.push_back(n);
    sort(ntCols.begin(), ntCols.end());

    vector<string> hdr = {"State"};
    for (auto &t : termCols)
        hdr.push_back(t);
    for (auto &n : ntCols)
        hdr.push_back(n);
}

```

```

vector<vector<string>> rows;
for (int i = 0; i < (int)STATES.size(); i++)
{
    vector<string> row = {to_string(i)};
    for (auto &t : termCols)
        row.push_back(ACT[i].count(t) ? ACT[i][t] : "-");
    for (auto &n : ntCols)
        row.push_back(GOT[i].count(n) ? to_string(GOT[i][n]) : "-");
    rows.push_back(row);
}

```

```

auto w = colWidths(hdr, rows);

```

```

int stateW = w[0];
int actionW = 0;
for (int i = 0; i < (int)termCols.size(); i++)
    actionW += w[1 + i] + 3;
actionW -= 1;
int gotoW = 0;
for (int i = 0; i < (int)ntCols.size(); i++)
    gotoW += w[1 + (int)termCols.size() + i] + 3;
gotoW -= 1;

```

```

cout << "+" << string(stateW + 2, '-')
    << "+" << string(actionW + 2, '-')
    << "+" << string(gotoW + 2, '-') << "+\n";
cout << "|" << left << setw(stateW) << "State"
    << " | " << left << setw(actionW) << "ACTION"
    << " | " << left << setw(gotoW) << "GOTO" << " |\n";

```

```

hline(w, '=');
printRow(w, hdr);
hline(w, '=');
for (auto &r : rows)
{
    printRow(w, r);
    hline(w, '-');
}

```

```

cout << "\n Legend:\n";
cout << "  sN = shift to state N\n";
cout << "  rN = reduce by production (N)\n";
cout << "  acc = accept\n";
cout << "  - = error / no entry\n";

```

```

cout << "\n Productions used in reduce actions:\n";
for (int i = 0; i < (int)G.size(); i++)
{
    cout << "    (" << i << ") " << G[i].first << " -> ";
    if (G[i].second.empty())
        cout << "eps";
    else
        for (auto &s : G[i].second)
            cout << s << " ";
    cout << "\n";
}
if (!conflicts.empty())
{
    cout << "\n [!] CONFLICTS (grammar may not be CLR(1)):\n";
    for (auto &c : conflicts)
        cout << "    " << c << "\n";
}
}

```

```

void parseInput()
{
    sectionBanner("PARSING INPUT: \"" + INPUT + "\"");

    string inp = INPUT + "$";
    int ip = 0;

    stack<int> stateStk;
    stack<string> symStk;
    stateStk.push(0);

    auto stackStr = [&]() -> string
    {
        vector<int> v;
        stack<int> tmp = stateStk;
        while (!tmp.empty())
        {
            v.push_back(tmp.top());
            tmp.pop();
        }
        reverse(v.begin(), v.end());
        string s;
        for (auto x : v)
            s += to_string(x) + " ";
        return s;
    }
}

```



```

};

auto symStr = [&]() -> string
{
    vector<string> v;
    stack<string> tmp = symStk;
    while (!tmp.empty())
    {
        v.push_back(tmp.top());
        tmp.pop();
    }
    reverse(v.begin(), v.end());
    string s = "$";
    for (auto &x : v)
        s += x;
    return s;
};

vector<string> hdr = {"Step", "State Stack", "Symbol Stack", "Input", "Action"};
vector<vector<string>> rows;

int step = 0;
bool accepted = false;
bool errored = false;

while (true)
{
    step++;
    int state = stateStk.top();
    string tok(1, inp[ip]);

    if (!ACT[state].count(tok))
    {
        rows.push_back({to_string(step), stackStr(), symStr(), inp.substr(ip), "ERROR"});
        errored = true;
        break;
    }

    string act = ACT[state][tok];

    if (act == "acc")
    {
        rows.push_back({to_string(step), stackStr(), symStr(), inp.substr(ip), "ACCEPT"});
        accepted = true;
    }
}

```

```

        break;
    }

    if (act[0] == 's')
    {
        int ns = stoi(act.substr(1));
        string desc = "Shift " + tok + " -> push s" + to_string(ns);
        rows.push_back({to_string(step), stackStr(), symStr(), inp.substr(ip), desc});
        stateStk.push(ns);
        symStk.push(tok);
        ip++;
    }
    else
    {
        int pid = stoi(act.substr(1));
        auto &[lhs, rhs] = G[pid];
        string prodStr = lhs + " -> ";
        for (auto &s : rhs)
            prodStr += " " + s;
        string desc = "Reduce by (" + to_string(pid) + ") " + prodStr;
        rows.push_back({to_string(step), stackStr(), symStr(), inp.substr(ip), desc});
        for (int i = 0; i < (int)rhs.size(); i++)
        {
            stateStk.pop();
            symStk.pop();
        }
        int top = stateStk.top();
        if (!GOT[top].count(lhs))
        {
            rows.push_back({to_string(step + 1), stackStr(), symStr(), inp.substr(ip), "GOTO
ERROR"});
            errored = true;
            break;
        }
        stateStk.push(GOT[top][lhs]);
        symStk.push(lhs);
    }
}

printTable(hdr, rows);

cout << "\n";
if (accepted)
    cout << " >> RESULT: \'" << INPUT << "\" is ACCEPTED by the grammar.\n\n";

```

```

else
    cout << " >> RESULT: \"\" << INPUT << "\" is REJECTED (syntax error at step "
        << step << ").\n\n";
}

int main()
{
    deriveSymbols();
    computeFirst();
    buildCollection();
    buildTables();

    printGrammar();
    printFirstSets();
    printStates();
    printParsingTable();
    parseInput();

    return 0;
}

```

Input and Output:

Input Grammar: (NT = {c , d}, T = {S, C }, P={ S -> CC, C -> cC, C -> d}, S = {S})

Input String: ccdd

```

PS C:\Users\vishw> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-9\" ; if ($?) { g++ clr_parser_main.cpp -o clr_parser_main } ; if ($?) { .\clr_parser_main }

=====
| GRAMMAR (Augmented) |
=====
(0) S' -> S
(1) S -> C C
(2) C -> c C
(3) C -> d

=====
| FIRST SETS |
=====
FIRST( C ) = { c, d }
FIRST( S ) = { c, d }

=====
| CANONICAL LR(1) ITEM SETS |
=====

State I0 -----
S' -> . S [$]
S -> . C C [$]
C -> . c C [c]
C -> . c C [d]
C -> . d [c]
C -> . d [d]

-----
goto( C ) => I1
goto( S ) => I2
goto( c ) => I3
goto( d ) => I4

```

```

State I1 -----
S -> C . C [$]
C -> . c C [$]
C -> . d [$]
-----

goto( C ) => I5
goto( c ) => I6
goto( d ) => I7


State I2 -----
S' -> S . [$]


State I3 -----
C -> . c C [c]
C -> . c C [d]
C -> c . C [c]
C -> c . C [d]
C -> . d [c]
C -> . d [d]
-----

goto( C ) => I8
goto( c ) => I3
goto( d ) => I4


State I4 -----
C -> d . [c]
C -> d . [d]


State I5 -----
S -> C C . [$]


State I6 -----
C -> . c C [$]
C -> c . C [$]
C -> . d [$]
-----

goto( C ) => I9
goto( c ) => I6
goto( d ) => I7

```

```

State I7 -----
  C -> d .  [$]

State I8 -----
  C -> c C .  [c]
  C -> c C .  [d]

State I9 -----
  C -> c C .  [$]

```

```

=====
| CLR(1)  PARSING  TABLE |
=====

```

State	ACTION					GOTO
State	c	d	\$	C	S	
0	s3	s4	-	1	2	
1	s6	s7	-	5	-	
2	-	-	acc	-	-	
3	s3	s4	-	8	-	
4	r3	r3	-	-	-	
5	-	-	r1	-	-	
6	s6	s7	-	9	-	
7	-	-	r3	-	-	
8	r2	r2	-	-	-	
9	-	-	r2	-	-	

Legend:

sN = shift to state N

rN = reduce by production (N)

acc = accept

- = error / no entry

Productions used in reduce actions:

(0) S' -> S

(1) S -> C C

(2) C -> c C

(3) C -> d

```
=====
| PARSING INPUT: "ccdd" |
=====

+=====+=====+=====+=====+=====+
| Step | State Stack | Symbol Stack | Input | Action |
+=====+=====+=====+=====+=====+
| 1 | 0 | $ | ccdd$ | Shift 'c' -> push s3 |
+-----+-----+-----+-----+-----+
| 2 | 0 3 | $c | cdd$ | Shift 'c' -> push s3 |
+-----+-----+-----+-----+-----+
| 3 | 0 3 3 | $cc | dd$ | Shift 'd' -> push s4 |
+-----+-----+-----+-----+-----+
| 4 | 0 3 3 4 | $ccd | d$ | Reduce by (3) C -> d |
+-----+-----+-----+-----+-----+
| 5 | 0 3 3 8 | $ccC | d$ | Reduce by (2) C -> c C |
+-----+-----+-----+-----+-----+
| 6 | 0 3 8 | $cC | d$ | Reduce by (2) C -> c C |
+-----+-----+-----+-----+-----+
| 7 | 0 1 | $C | d$ | Shift 'd' -> push s7 |
+-----+-----+-----+-----+-----+
| 8 | 0 1 7 | $Cd | $ | Reduce by (3) C -> d |
+-----+-----+-----+-----+-----+
| 9 | 0 1 5 | $CC | $ | Reduce by (1) S -> C C |
+-----+-----+-----+-----+-----+
| 10 | 0 2 | $S | $ | ACCEPT |
+-----+-----+-----+-----+-----+

>> RESULT: "ccdd" is ACCEPTED by the grammar.
```

Result:

The CLR parser has been successfully written in C++ and executes as expected.

Compiler Design Lab

Date: 6-3-2026

Assignment No.: 9

Name: S Vishwajith

Register No.: 23BCE1145

Aim:

To implement a code to find Three Address code and it's 3 data structures using C++.

Code:

```
#include <iostream>
#include <stack>
#include <vector>

using namespace std;
struct token
{
    string c;
    int type; // 0-id, 1-unary, 2-binary, 3-equals
};
struct statement
{
    int temp; // 0-none, 1-arg1, 2-arg2, 3-arg1&&arg2
    int eq;
    string label;
    string res;
    string op;
    string arg1;
    string arg2;
};

vector<token> input;
vector<statement> tac;
int count = 0;
int ifcount = 0;

void to_tac()
{
    stack<token> s;
    stack<int> ifstack;
```

```

string lab = "";
int ind = 0;
for (token t : input)
{
    ind++;
    if (t.type == 0 && t.c == "if")
    {
        statement st;
        st.op = "if True goto";
        st.arg1 = s.top().c;
        st.temp = 0;
        st.eq = 2;
        if (st.arg1[0] == 't' && st.arg1[1] <= '9' && st.arg1[1] >= '0')
            st.temp = 1;
        st.arg2 = "L" + to_string(ifcount);
        lab = "L" + to_string(ifcount++);
        st.res = "";
        tac.push_back(st);
        statement st1;
        st1.eq = 3;
        st1.temp = 0;
        st1.op = "goto";
        st1.arg1 = "L";
        st1.arg2 = "";
        st1.res = "";
        tac.push_back(st1);
        ifstack.push(tac.size() - 1);
    }
    else if (t.type == 0 && t.c == "else")
    {
        statement st1;
        st1.eq = 3;
        st1.temp = 0;
        st1.op = "goto";
        st1.arg1 = "L";
        st1.arg2 = "";
        st1.res = "";
        tac.push_back(st1);
        lab = "L" + to_string(ifcount);
        tac[ifstack.top()].arg1 += to_string(ifcount++);
        ifstack.pop();
        ifstack.push(tac.size() - 1);
    }
    else if (t.type == 0 && t.c == "endif")

```



```

{
    lab = "L" + to_string(ifcount);
    if (ind == input.size())
    {
        statement st;
        st.eq = 5;
        st.label = "L" + to_string(ifcount);
        tac.push_back(st);
        while (!s.empty())
        {
            string lb = tac[ifstack.top()].arg1;
            if (lb == "L")
                tac[ifstack.top()].arg1 += to_string(ifcount);
            s.pop();
        }
    }
}
else if (t.type == 0)
    s.push(t);
else if (t.type == 1)
{
    token arg = s.top();
    s.pop();
    statement st;
    st.temp = 0;
    st.eq = 0;
    st.res = "t" + to_string(::count);
    st.op = t.c;
    st.arg1 = arg.c;
    st.arg2 = "";
    if (st.arg1[0] == 't' && st.arg1[1] <= '9' && st.arg1[1] >= '0')
        st.temp = 1;
    if (lab != "")
    {
        st.label = lab;
        lab = "";
    }
    tac.push_back(st);
    token n;
    n.type = 0;
    n.c = "t" + to_string(::count++);
    s.push(n);
}
else if (t.type == 2)

```

```

{
    token arg2 = s.top();
    s.pop();
    token arg1 = s.top();
    s.pop();
    statement st;
    st.temp = 0;
    st.eq = 0;
    st.res = "t" + to_string(::count);
    st.op = t.c;
    st.arg1 = arg1.c;
    st.arg2 = arg2.c;
    if (st.arg1[0] == 't' && st.arg1[1] <= '9' && st.arg1[1] >= '0')
        st.temp += 1;
    if (st.arg2[0] == 't' && st.arg2[1] <= '9' && st.arg2[1] >= '0')
        st.temp += 2;
    if (lab != "")
    {
        st.label = lab;
        lab = "";
    }
    tac.push_back(st);
    token n;
    n.type = 0;
    n.c = "t" + to_string(::count++);
    s.push(n);
}
else if (t.type == 3)
{
    token arg1 = s.top();
    s.pop();
    token arg2 = s.top();
    s.pop();
    statement st;
    st.arg1 = arg2.c;
    st.op = "=";
    st.arg2 = arg1.c;
    st.eq = 1;
    st.res = "";
    if (lab != "")
    {
        st.label = lab;
        lab = "";
    }
}

```

```

        tac.push_back(st);
    }
}
}

```

```

int find_temp(string temp)
{
    int i = 0;
    for (statement s : tac)
    {
        if (s.res == temp || (s.arg1 == temp && s.eq == 1))
            return i;
        i++;
    }
    return -1;
}

```

```

void print_tac()
{
    cout << endl
        << "Three Address Code: " << endl;
    for (statement s : tac)
    {
        if (s.label == "")
            cout << " ";
        else
            cout << s.label << ": ";
        if (s.eq == 1)
            cout << s.arg1 << " = " << s.arg2;
        else if (s.eq == 2)
            cout << "if " << s.arg1 << " True goto " << s.arg2;
        else if (s.eq == 3)
            cout << "goto " << s.arg1;
        else if (s.eq == 5)
            cout << " ";
        else if (s.arg2 == "")
            cout << s.res << " = " << s.op << " " << s.arg1;
        else
            cout << s.res << " = " << s.arg1 << " " << s.op << " " << s.arg2;
        cout << endl;
    }
}

```

```

void print_quad()

```

```

{
    cout << "\nQuadruples: \n\n";
    cout << "|   OP   | ARG1 | ARG2 | RES |" << endl;
    cout << "-----" << endl;
    for (statement s : tac)
    {
        if (s.eq == 5)
            continue;
        string pad;
        cout << '|';
        string out = s.op;
        pad = string(12 - out.size(), ' ');
        cout << out << pad << '|';
        out = s.arg1;
        pad = string(10 - out.size(), ' ');
        cout << out << pad << '|';
        out = s.arg2;
        pad = string(10 - out.size(), ' ');
        cout << out << pad << '|';
        out = s.res;
        pad = string(10 - out.size(), ' ');
        cout << out << pad << '|';
        cout << endl;
    }
}

void print_trip()
{
    cout << "\nTriples: \n\n";
    cout << "| INDEX |   OP   | ARG1 | ARG2 |" << endl;
    cout << "-----" << endl;
    int i = 0;
    for (statement s : tac)
    {
        if (s.eq == 5)
            continue;
        string pad;
        string out;
        cout << '|';
        out = to_string(i);
        pad = string(12 - out.size(), ' ');
        cout << out << pad << '|';
        out = s.op;
        pad = string(12 - out.size(), ' ');

```

```

        cout << out << pad << '|';
        out = s.arg1;
        if (out[0] == 't' && out.size() > 1)
            out = '[' + to_string(find_temp(out)) + ']';
        pad = string(10 - out.size(), ' ');
        cout << out << pad << '|';
        out = s.arg2;
        if (out[0] == 't' && out.size() > 1)
            out = '[' + to_string(find_temp(out)) + ']';
        pad = string(10 - out.size(), ' ');
        cout << out << pad << '|';
        cout << endl;
        i++;
    }
}

void print_i_trip()
{
    cout << "\nIndirect Triples: \n\n";
    cout << "| ADDR | INDEX | | INDEX | OP | ARG1 | ARG2 |" << endl;
    cout << "-----" << endl;
    int i = 0;
    int a = 31;
    for (statement s : tac)
    {
        if (s.eq == 5)
            continue;
        string pad;
        string out;
        cout << '|';
        out = to_string(a);
        pad = string(8 - out.size(), ' ');
        cout << out << pad << '|';
        out = to_string(i);
        pad = string(12 - out.size(), ' ');
        cout << out << pad << '|';
        cout << " |";
        out = to_string(i);
        pad = string(12 - out.size(), ' ');
        cout << out << pad << '|';
        out = s.op;
        pad = string(12 - out.size(), ' ');
        cout << out << pad << '|';
        out = s.arg1;

```

```

        if (out[0] == 't' && out.size() > 1)
            out = '[' + to_string(a - i + find_temp(out)) + ']';
        pad = string(10 - out.size(), ' ');
        cout << out << pad << '|';
        out = s.arg2;
        if (out[0] == 't' && out.size() > 1)
            out = '[' + to_string(a - i + find_temp(out)) + ']';
        pad = string(10 - out.size(), ' ');
        cout << out << pad << '|';
        cout << endl;
        i++;
        a++;
    }
}

```

```

int main()
{
    string in, var;
    int un = 0;
    cin >> in;
    cout << "INPUT: " << in << endl;
    for (char c : in)
    {
        if (c == ',')
        {
            if (!var.empty())
            {
                token n;
                n.type = 0;
                n.c = var;
                input.push_back(n);
                var.clear();
            }
            un = 0;
        }
        else if (c == 'u')
        {
            un = 1;
            var.push_back(c);
        }
        else if (c == '-' && un == 1)
        {
            token n;
            n.type = 1;

```

```

        n.c = '-';
        input.push_back(n);
        var.clear();
        un = 0;
    }
    else if (c == '=')
    {
        token n;
        n.type = 3;
        n.c = '=';
        input.push_back(n);
        un = 0;
        var.clear();
    }
    else if (('a' <= c && c <= 'z') || ('0' <= c && c <= '9') || ('A' <= c && c <= 'Z'))
    {
        var.push_back(c);
        un = 0;
    }
    else if (c == '+' || c == '-' || c == '%' || c == '*' || c == '/' || c == '^' || c == '<')
    {
        token n;
        n.type = 2;
        n.c = c;
        input.push_back(n);
        var.clear();
        un = 0;
    }
}
if (!var.empty())
{
    token n;
    n.type = 0;
    n.c = var;
    input.push_back(n);
    var.clear();
}

to_tac();
print_tac();
print_quad();
print_trip();
print_i_trip();
}

```

Inputs and Outputs:

Input 1: $p = q * -r + q - r + (t * u + v * w)$

Postfix: $p, q, r, u, -, *, q, +, r, -, t, u, *, v, w, *, +, +, =$

Output:

```
PS C:\Users\vishw\Coding\Compiler-Lab\Week-10> .\TAC
```

```
p,q,r,u-,*,q,+,r,-,t,u,*,v,w,*,+,+,=
```

```
INPUT: p,q,r,u-,*,q,+,r,-,t,u,*,v,w,*,+,+,=
```

Three Address Code:

```
t0 = - r
t1 = q * t0
t2 = t1 + q
t3 = t2 - r
t4 = t * u
t5 = v * w
t6 = t4 + t5
t7 = t3 + t6
p = t7
```

Quadruples:

OP	ARG1	ARG2	RES
-	r		t0
*	q	t0	t1
+	t1	q	t2
-	t2	r	t3
*	t	u	t4
*	v	w	t5
+	t4	t5	t6
+	t3	t6	t7
=	p	t7	

Triples:				
	INDEX	OP	ARG1	ARG2
0		-	r	
1		*	q	[0]
2		+	[1]	q
3		-	[2]	r
4		*	t	u
5		*	v	w
6		+	[4]	[5]
7		+	[3]	[6]
8		=	p	[7]

Indirect Triples:							
	ADDR	INDEX		INDEX	OP	ARG1	ARG2
31	0		0		-	r	
32	1		1		*	q	[31]
33	2		2		+	[32]	q
34	3		3		-	[33]	r
35	4		4		*	t	u
36	5		5		*	v	w
37	6		6		+	[35]	[36]
38	7		7		+	[34]	[37]
39	8		8		=	p	[38]

Input 2: x = (m+(n^o) / ((p-q)/(r+(n^o)) % s/t))

Postfix: x,m,n,o,^,p,q,-,r,n,o,^,+,/,s,t,/,%,/,=

Output:

x,m,n,o,^,p,q,-,r,n,o,^,+,/,s,t,/,%,/,=
 INPUT: x,m,n,o,^,p,q,-,r,n,o,^,+,/,s,t,/,%,/,=

Three Address Code:

```

t0 = n ^ o
t1 = p - q
t2 = n ^ o
t3 = r + t2
t4 = t1 / t3
t5 = s / t
t6 = t4 % t5
t7 = t0 / t6
m = t7

```

Quadruples:

OP	ARG1	ARG2	RES
^	n	o	t0
-	p	q	t1
^	n	o	t2
+	r	t2	t3
/	t1	t3	t4
/	s	t	t5
%	t4	t5	t6
/	t0	t6	t7
=	m	t7	

Triples:				
	INDEX	OP	ARG1	ARG2
0		^	n	o
1		−	p	q
2		^	n	o
3		+	r	[2]
4		/	[1]	[3]
5		/	s	t
6		%	[4]	[5]
7		/	[0]	[6]
8		=	m	[7]

Indirect Triples:							
	ADDR	INDEX		INDEX	OP	ARG1	ARG2
31	0		0		^	n	o
32	1		1		−	p	q
33	2		2		^	n	o
34	3		3		+	r	[33]
35	4		4		/	[32]	[34]
36	5		5		/	s	t
37	6		6		%	[35]	[36]
38	7		7		/	[31]	[37]
39	8		8		=	m	[38]

Input 3: if x<y, p=p+1 else q=q+1 endif
Postfix: x,y,<,if,p,p,1,+,=,else,q,q,1,+,=,endif
Output:

```
PS C:\Users\vishw\Coding\Compiler-Lab\Week-10> .\TAC
x,y,<,if,p,p,1,+,=,else,q,q,1,+,=,endif
INPUT: x,y,<,if,p,p,1,+,=,else,q,q,1,+,=,endif
```

Three Address Code:

```
    t0 = x < y
    if t0 True goto L0
    goto L1
L0: t1 = p + 1
    p = t1
    goto L2
L1: t2 = q + 1
    q = t2
L2:
```

Quadruples:

OP	ARG1	ARG2	RES
<	x	y	t0
if True goto	t0	L0	
goto	L1		
+	p	1	t1
=	p	t1	
goto	L2		
+	q	1	t2
=	q	t2	

Triples:

INDEX	OP	ARG1	ARG2
0	<	x	y
1	if True goto	[0]	L0
2	goto	L1	
3	+	p	1
4	=	p	[3]
5	goto	L2	
6	+	q	1
7	=	q	[6]

Indirect Triples:

ADDR	INDEX	INDEX	OP	ARG1	ARG2
31	0	0	<	x	y
32	1	1	if True goto	[31]	L0
33	2	2	goto	L1	
34	3	3	+	p	1
35	4	4	=	p	[34]
36	5	5	goto	L2	
37	6	6	+	q	1
38	7	7	=	q	[37]

PS C:\Users\vishw\Coding\Compiler-Lab\Week-10> |

Result:

We have successfully implemented a C++ code to find Three Address Code and it's 3 data structures (quadruple, triple, indirect triple).

Compiler Design Lab

Date: 13-3-2026

Assignment No.: 10

Name: S Vishwajith

Register No.: 23BCE1145

Aim:

To implement a code to find Three Address code and represent it in quadruples and triples using C++.

Code:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <map>
#include <set>
#include <stack>
#include <iomanip>
#include <cstdlib>
using namespace std;
```

```
enum TokKind
```

```
{
    T_ID,
    T_NUM,
    T_ASSIGN,
    T_PLUS,
    T_MINUS,
    T_STAR,
    T_SLASH,
    T_LT,
    T_GT,
    T_LE,
    T_GE,
    T_EQ,
    T_NEQ,
    T_AND,
    T_OR,
```

```

T_NOT,
T_LPAREN,
T_RPAREN,
T_LBRACKET,
T_RBRACKET,
T_LBRACE,
T_RBRACE,
T_SEMI,
T_COMMA,
T_IF,
T_ELSE,
T_WHILE,
T_FOR,
T_BREAK,
T_EOF
};

struct Token
{
    TokKind kind;
    string val;
    int line;
};

class Lexer
{
    string src;
    size_t pos = 0;
    int line = 1;

    bool peek2(char a, char b) const
    {
        return pos + 1 < src.size() && src[pos] == a && src[pos + 1] == b;
    }

    void skipWS()
    {
        while (pos < src.size())
        {
            char c = src[pos];
            if (c == '\n')
            {
                ++line;
                ++pos;
            }
        }
    }
};

```

```

    }
    else if (isspace((unsigned char)c))
    {
        ++pos;
    }
    else if (peek2('/', '/'))
    {
        while (pos < src.size() && src[pos] != '\n')
            ++pos;
    }
    else
        break;
}
}

```

```

static TokKind keyword(const string &s)
{
    if (s == "if")
        return T_IF;
    if (s == "else")
        return T_ELSE;
    if (s == "while")
        return T_WHILE;
    if (s == "for")
        return T_FOR;
    if (s == "break")
        return T_BREAK;
    return T_ID;
}

```

```

public:
    explicit Lexer(string source) : src(move(source)) {}

```

```

    Token next()
    {
        skipWS();
        if (pos >= src.size())
            return {T_EOF, "", line};

        char c = src[pos];

        if (isdigit((unsigned char)c))
        {
            string num;

```



```

    while (pos < src.size() && isdigit((unsigned char)src[pos]))
        num += src[pos++];
    return {T_NUM, num, line};
}

if (isalpha((unsigned char)c) || c == '_')
{
    string id;
    while (pos < src.size() &&
        (isalnum((unsigned char)src[pos]) || src[pos] == '_'))
        id += src[pos++];
    return {keyword(id), id, line};
}

if (peek2('<', '='))
{
    pos += 2;
    return {T_LE, "<=", line};
}
if (peek2('>', '='))
{
    pos += 2;
    return {T_GE, ">=", line};
}
if (peek2('=', '='))
{
    pos += 2;
    return {T_EQ, "==", line};
}
if (peek2('!', '='))
{
    pos += 2;
    return {T_NEQ, "!=", line};
}
if (peek2('&', '&'))
{
    pos += 2;
    return {T_AND, "&&", line};
}
if (peek2('|', '|'))
{
    pos += 2;
    return {T_OR, "||", line};
}

```

```

++pos;
switch (c)
{
case '=':
    return {T_ASSIGN, "=", line};
case '+':
    return {T_PLUS, "+", line};
case '-':
    return {T_MINUS, "-", line};
case '*':
    return {T_STAR, "*", line};
case '/':
    return {T_SLASH, "/", line};
case '<':
    return {T_LT, "<", line};
case '>':
    return {T_GT, ">", line};
case '!':
    return {T_NOT, "!", line};
case '(':
    return {T_LPAREN, "(", line};
case ')':
    return {T_RPAREN, ")", line};
case '[':
    return {T_LBRACKET, "[", line};
case ']':
    return {T_RBRACKET, "]", line};
case '{':
    return {T_LBRACE, "{", line};
case '}':
    return {T_RBRACE, "}", line};
case ';':
    return {T_SEMI, ";", line};
case ',':
    return {T_COMMA, ",", line};
default:
    cerr << "Unknown character '" << c
        << "' at line " << line << '\n';
    return {T_EOF, "", line};
}
}

```

```
vector<Token> tokenise()
```

```

{
    vector<Token> v;
    Token t;
    do
    {
        t = next();
        v.push_back(t);
    } while (t.kind != T_EOF);
    return v;
}
};

```

```

struct Instr
{
    string label;
    string op, arg1, arg2, result;

    string str() const
    {
        if (op == "goto")
            return "goto " + result;
        if (op == "if False")
            return "if False " + arg1 + " goto " + result;
        if (op == "=")
            return result + " = " + arg1;
        if (op == "[r]")
            return result + " = " + arg1 + "[" + arg2 + "]";
        if (op == "[w]")
            return arg1 + "[" + arg2 + "] = " + result;
        if (!arg2.empty())
            return result + " = " + arg1 + " " + op + " " + arg2;
        if (!arg1.empty())
            return result + " = " + op + " " + arg1;
        return op;
    }
};

```

```

class CodeGen
{
    vector<Token> toks;
    size_t cur = 0;
    vector<Instr> code;
    int tempCnt = 0;
    int labelCnt = 0;

```

```

stack<string> breakTarget; // innermost loop exit label

Token &peek() { return toks[cur]; }
bool at(TokKind k) { return peek().kind == k; }
Token consume() { return at(T_EOF) ? toks[cur] : toks[cur++]; }
bool match(TokKind k)
{
    if (at(k))
    {
        consume();
        return true;
    }
    return false;
}

void expect(TokKind k, const char *what)
{
    if (!match(k))
    {
        cerr << "Parse error at line " << peek().line
              << ": expected " << what
              << ", got '" << peek().val << "'\n";
        exit(1);
    }
}

string freshTemp() { return "t" + to_string(++tempCnt); }
string freshLabel() { return "L" + to_string(++labelCnt); }

void emit(const string &op,
          const string &a1 = "",
          const string &a2 = "",
          const string &res = "")
{
    code.push_back({"", op, a1, a2, res});
}

void emitLabel(const string &lbl)
{
    code.push_back({lbl, "label", "", "", ""});
}

string parseExpr() { return parseOr(); }

string parseOr()

```

```

{
    string lhs = parseAnd();
    while (at(T_OR))
    {
        consume();
        string rhs = parseAnd();
        string t = freshTemp();
        emit("||", lhs, rhs, t);
        lhs = t;
    }
    return lhs;
}

```

```

string parseAnd()
{
    string lhs = parseRel();
    while (at(T_AND))
    {
        consume();
        string rhs = parseRel();
        string t = freshTemp();
        emit("&&", lhs, rhs, t);
        lhs = t;
    }
    return lhs;
}

```

```

string parseRel()
{
    string lhs = parseAdd();
    for (;;)
    {
        string op;
        switch (peek().kind)
        {
            case T_LT:
                op = "<";
                break;
            case T_GT:
                op = ">";
                break;
            case T_LE:
                op = "<=";
                break;

```

```

    case T_GE:
        op = ">=";
        break;
    case T_EQ:
        op = "==";
        break;
    case T_NEQ:
        op = "!=";
        break;
    default:
        return lhs;
    }
    consume();
    string rhs = parseAdd();
    string t = freshTemp();
    emit(op, lhs, rhs, t);
    lhs = t;
}
}

string parseAdd()
{
    string lhs = parseMul();
    while (at(T_PLUS) || at(T_MINUS))
    {
        string op = consume().val;
        string rhs = parseMul();
        string t = freshTemp();
        emit(op, lhs, rhs, t);
        lhs = t;
    }
    return lhs;
}

string parseMul()
{
    string lhs = parseUnary();
    while (at(T_STAR) || at(T_SLASH))
    {
        string op = consume().val;
        string rhs = parseUnary();
        string t = freshTemp();
        emit(op, lhs, rhs, t);
        lhs = t;
    }
}

```

```

    }
    return lhs;
}

string unary()
{
    if (at(T_MINUS))
    {
        consume();
        string t = freshTemp();
        emit("-", parsePrimary(), "", t);
        return t;
    }
    if (at(T_NOT))
    {
        consume();
        string t = freshTemp();
        emit("!", parsePrimary(), "", t);
        return t;
    }
    return parsePrimary();
}

```

```

string parsePrimary()
{
    if (at(T_LPAREN))
    {
        consume();
        string v = parseExpr();
        expect(T_RPAREN, "");
        return v;
    }
    if (at(T_NUM))
        return consume().val;

    if (at(T_ID))
    {
        string name = consume().val;

        if (at(T_LBRACKET))
        {
            consume();
            string idx1 = parseExpr();
            expect(T_RBRACKET, "");

```

```

    if (at(T_LBRACKET))
    {
        consume();
        string idx2 = parseExpr();
        expect(T_RBRACKET, "]");

        string tRow = freshTemp();
        string tOff = freshTemp();
        string tVal = freshTemp();
        emit(".*", idx1, "cols", tRow);
        emit("+", tRow, idx2, tOff);
        emit("[r", name, tOff, tVal);
        return tVal;
    }

    string t = freshTemp();
    emit("[r", name, idx1, t);
    return t;
}
return name;
}

cerr << "Unexpected token " << peek().val
    << " at line " << peek().line << "\n";
exit(1);
}

void parseBlock()
{
    if (at(T_LBRACE))
    {
        consume();
        while (!at(T_RBRACE) && !at(T_EOF))
            parseStmt();
        expect(T_RBRACE, "}");
    }
    else
    {
        parseStmt();
    }
}

void parseIf()

```



```

{
    consume();
    expect(T_LPAREN, "(");
    string cond = parseExpr();
    expect(T_RPAREN, ");

    string IFalse = freshLabel();
    string IEnd = freshLabel();

    emit("if False", cond, "", IFalse);
    parseBlock();

    if (at(T_ELSE))
    {
        consume();
        emit("goto", "", "", IEnd);
        emitLabel(IFalse);
        if (at(T_IF))
            parself();
        else
            parseBlock();
        emitLabel(IEnd);
    }
    else
    {
        emitLabel(IFalse);
    }
}

void parseWhile()
{
    consume();

    string IStart = freshLabel();
    string IEnd = freshLabel();

    breakTarget.push(IEnd);

    emitLabel(IStart);
    expect(T_LPAREN, "(");
    string cond = parseExpr();
    expect(T_RPAREN, ");

    emit("if False", cond, "", IEnd);

```

```

    parseBlock();
    emit("goto", "", "", IStart);
    emitLabel(LEnd);

    breakTarget.pop();
}

void parseFor()
{
    consume();
    expect(T_LPAREN, "(");

    if (!at(T_SEMI))
        parseAssign();
    expect(T_SEMI, ";");

    string lCond = freshLabel();
    string lBody = freshLabel();
    string lIncr = freshLabel();
    string lEnd = freshLabel();

    breakTarget.push(lEnd);

    emitLabel(lCond);

    string cond = "1";
    if (!at(T_SEMI))
        cond = parseExpr();
    expect(T_SEMI, ";");

    emit("if False", cond, "", lEnd);
    emit("goto", "", "", lBody);

    size_t incrStart = cur;
    {
        int depth = 1;
        while (cur < toks.size() && depth > 0)
        {
            if (toks[cur].kind == T_LPAREN)
                ++depth;
            if (toks[cur].kind == T_RPAREN)
                --depth;
            ++cur;
        }
    }
}

```

```

    }

    emitLabel(lBody);
    parseBlock();

    emitLabel(lIncr);
    size_t afterBody = cur;
    cur = incrStart;
    if (!at(T_RPAREN))
        parseAssign();
    cur = afterBody;

    emit("goto", "", "", lCond);
    emitLabel(lEnd);

    breakTarget.pop();
}

void parseAssign()
{
    if (!at(T_ID))
    {
        parseExpr();
        return;
    }

    string name = consume().val;

    if (at(T_LBRACKET))
    {
        consume();
        string idx1 = parseExpr();
        expect(T_RBRACKET, "]");

        if (at(T_LBRACKET))
        {
            consume();
            string idx2 = parseExpr();
            expect(T_RBRACKET, "]");
            expect(T_ASSIGN, "=");
            string rhs = parseExpr();

            string tRow = freshTemp();
            string tOff = freshTemp();

```

```

        emit("x", idx1, "cols", tRow);
        emit("+", tRow, idx2, tOff);
        emit("[]w", name, tOff, rhs);
        return;
    }

    expect(T_ASSIGN, "=");
    string rhs = parseExpr();
    emit("[]w", name, idx1, rhs);
    return;
}

if (at(T_ASSIGN))
{
    consume();
    string rhs = parseExpr();
    emit("=", rhs, "", name);
    return;
}

--cur;
parseExpr();
}

void parseStmt()
{
    switch (peek().kind)
    {
    case T_IF:
        parself();
        break;
    case T_WHILE:
        parseWhile();
        break;
    case T_FOR:
        parseFor();
        break;
    case T_BREAK:
        consume();
        if (!breakTarget.empty())
            emit("goto", "", "", breakTarget.top());
        else
            cerr << "Warning: 'break' outside loop at line "
                << peek().line << '\n';
    }
}

```

```

        match(T_SEMI);
        break;
    case T_LBRACE:
        parseBlock();
        break;
    case T_SEMI:
        consume();
        break;
    default:
        parseAssign();
        match(T_SEMI);
        break;
    }
}

```

```

public:
    explicit CodeGen(vector<Token> tokens) : toks(move(tokens)) {}

```

```

    void run()
    {
        while (!at(T_EOF))
            parseStmt();
    }

```

```

    const vector<Instr> &instructions() const { return code; }
};

```

```

static string pad(const string &s, int w)
{
    string r = s;
    if ((int)r.size() > w)
        r = r.substr(0, w);
    while ((int)r.size() < w)
        r += ' ';
    return r;
}

```

```

static void hRule(int style, const vector<int> &widths)
{
    const char *L[] = {"+", "+", "+"};
    const char *M[] = {"+", "+", "+"};
    const char *R[] = {"+", "+", "+"};
    cout << L[style];
    for (size_t i = 0; i < widths.size(); ++i)

```

```

    {
        for (int j = 0; j < widths[i] + 2; ++j)
            cout << "-";
        cout << (i + 1 < widths.size() ? M[style] : R[style]);
    }
    cout << '\n';
}

static void printTAC(const vector<Instr> &code)
{
    cout <<
    "\n+=====+\n"
        "|          THREE-ADDRESS CODE (Labelled Form)          |\n"
        "+=====+\n\n";

    int seq = 0;
    for (const auto &ins : code)
    {
        if (ins.op == "label")
        {
            if (seq > 0)
                cout << '\n';
            cout << ins.label << ":\n";
        }
        else
        {
            cout << " [" << setw(3) << setfill('0') << seq
                << setfill(' ') << "]" << ins.str() << '\n';
            ++seq;
        }
    }
    cout << '\n';
}

static void printQuadruple(const vector<Instr> &code)
{
    cout <<
    "\n+=====+\n"
        "|          QUADRUPLE TABLE          |\n"
        "+=====+\n";

    const int WI = 4, WO = 10, WA = 14, WB = 14, WR = 16;
    hRule(0, {WI, WO, WA, WB, WR});

```

```

cout << "|" << pad("#", WI) << " | "
    << pad("Op", WO) << " | "
    << pad("Arg1", WA) << " | "
    << pad("Arg2", WB) << " | "
    << pad("Result", WR) << " |\n";
hRule(1, {WI, WO, WA, WB, WR});

int n = 0;
for (const auto &ins : code)
{
    if (ins.op == "label")
        continue;
    cout << "|" << pad(to_string(n++), WI) << " | "
        << pad(ins.op, WO) << " | "
        << pad(ins.arg1, WA) << " | "
        << pad(ins.arg2, WB) << " | "
        << pad(ins.result, WR) << " |\n";
}
hRule(2, {WI, WO, WA, WB, WR});
cout << "\n";
}

struct Triple
{
    string op, arg1, arg2;
};

static string resolveTriple(const string &arg,
                           const map<string, int> &t2i)
{
    auto it = t2i.find(arg);
    return (it != t2i.end()) ? "(" + to_string(it->second) + ")" : arg;
}

static void printTriple(const vector<Instr> &code)
{
    cout <<
    "\n+=====+\n"
        "|          TRIPLE TABLE          |\n"
        "+=====+\n";

    vector<Triple> triples;
    map<string, int> t2i;

```

```

for (const auto &ins : code)
{
    if (ins.op == "label")
        continue;

    Triple tr;
    tr.op = ins.op;

    if (ins.op == "=")
    {
        tr.arg1 = resolveTriple(ins.arg1, t2i);
        tr.arg2 = resolveTriple(ins.result, t2i);
    }
    else if (ins.op == "goto")
    {
        tr.arg1 = ins.result;
        tr.arg2 = "";
    }
    else if (ins.op == "if False")
    {
        tr.arg1 = resolveTriple(ins.arg1, t2i);
        tr.arg2 = ins.result;
    }
    else if (ins.op == "[w]")
    {
        tr.arg1 = ins.arg1 + "[" + resolveTriple(ins.arg2, t2i) + "]";
        tr.arg2 = resolveTriple(ins.result, t2i);
    }
    else if (ins.arg2.empty() && !ins.arg1.empty())
    {
        tr.arg1 = resolveTriple(ins.arg1, t2i);
        tr.arg2 = resolveTriple(ins.result, t2i);
    }
    else
    {
        tr.arg1 = resolveTriple(ins.arg1, t2i);
        tr.arg2 = resolveTriple(ins.arg2, t2i);
    }

    int idx = (int)triples.size();
    triples.push_back(tr);

    if (!ins.result.empty() && ins.result[0] == 't' && isdigit((unsigned char)ins.result[1]))
        t2i[ins.result] = idx;
}

```



```

}

const int WI = 4, WO = 10, WA = 18, WB = 18;
hRule(0, {WI, WO, WA, WB});
cout << "|" << pad("#", WI) << " | "
    << pad("Op", WO) << " | "
    << pad("Arg1", WA) << " | "
    << pad("Arg2", WB) << " \n";
hRule(1, {WI, WO, WA, WB});

for (int i = 0; i < (int)triples.size(); ++i)
{
    cout << "|" << pad(to_string(i), WI) << " | "
        << pad(triples[i].op, WO) << " | "
        << pad(triples[i].arg1, WA) << " | "
        << pad(triples[i].arg2, WB) << " \n";
}
hRule(2, {WI, WO, WA, WB});
cout << "\n";
}

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        cout << "Give the input file name next to the execution file." << endl;
        return 1;
    }

    ifstream fin(argv[1]);
    if (!fin)
    {
        cerr << "Error: cannot open " << argv[1] << "\n";
        return 1;
    }

    string src((istreambuf_iterator<char>(fin)),
               istreambuf_iterator<char>());

    cout << "\nSource file : " << argv[1] << "\n";
    cout << "-----\n";
    cout << src;
    if (!src.empty() && src.back() != '\n')
        cout << "\n";
}

```

```

cout << "-----\n";

Lexer lex(src);
auto tokens = lex.tokenise();

CodeGen gen(move(tokens));
gen.run();
const auto &code = gen.instructions();

if (code.empty())
{
    cerr << "\nNothing to generate -- check the input file for syntax errors.\n";
    return 1;
}

printTAC(code);
printQuadruple(code);
printTriple(code);

return 0;
}

```

Inputs and Outputs:

Code 1:

```
PS C:\Users\vishw\Coding\Compiler-Lab\Week-11> .\TAC.exe input1.txt
```

```
Source file : input1.txt
```

```
-----  
i = 0  
sum = 0  
fail_count = 0  
while (i < n)  
{  
    if (marks[i] > 50)  
        sum = sum + marks[i]  
    else  
        fail_count = fail_count + 1  
    i = i + 1  
}  
-----
```

```
+=====+  
|          THREE-ADDRESS CODE   (Labelled Form)          |  
+=====+
```

```
[000]    i = 0  
[001]    sum = 0  
[002]    fail_count = 0
```

```
L1:
```

```
[003]    t1 = i < n  
[004]    if False t1 goto L2  
[005]    t2 = marks[i]  
[006]    t3 = t2 > 50  
[007]    if False t3 goto L3  
[008]    t4 = marks[i]  
[009]    t5 = sum + t4  
[010]    sum = t5  
[011]    goto L4
```

```
L3:
```

```
[012]    t6 = fail_count + 1  
[013]    fail_count = t6
```

```
L4:
```

```
[014]    t7 = i + 1  
[015]    i = t7  
[016]    goto L1
```

```
L2:
```

QUADRUPLE TABLE				
#	Op	Arg1	Arg2	Result
0	=	0		i
1	=	0		sum
2	=	0		fail_count
3	<	i	n	t1
4	if False	t1		L2
5	[]r	marks	i	t2
6	>	t2	50	t3
7	if False	t3		L3
8	[]r	marks	i	t4
9	+	sum	t4	t5
10	=	t5		sum
11	goto			L4
12	+	fail_count	1	t6
13	=	t6		fail_count
14	+	i	1	t7
15	=	t7		i
16	goto			L1

TRIPLE TABLE			
#	Op	Arg1	Arg2
0	=	0	i
1	=	0	sum
2	=	0	fail_count
3	<	i	n
4	if False	(3)	L2
5	[]r	marks	i
6	>	(5)	50
7	if False	(6)	L3
8	[]r	marks	i
9	+	sum	(8)
10	=	(9)	sum
11	goto	L4	
12	+	fail_count	1
13	=	(12)	fail_count
14	+	i	1
15	=	(14)	i
16	goto	L1	

Code 2:

```
PS C:\Users\vishw\Coding\Compiler-Lab\Week-11> .\TAC.exe input2.txt
```

```
Source file : input2.txt
```

```
-----  
if (marks > 80 && attendance > 90)  
    scholarship = 2  
else if (marks > 70 || sports == 1)  
    scholarship = 1  
else  
    scholarship = 0  
-----
```

```
+=====+  
|          THREE-ADDRESS CODE (Labelled Form)          |  
+=====+
```

```
[000]    t1 = marks > 80  
[001]    t2 = attendance > 90  
[002]    t3 = t1 && t2  
[003]    if False t3 goto L1  
[004]    scholarship = 2  
[005]    goto L2
```

```
L1:  
[006]    t4 = marks > 70  
[007]    t5 = sports == 1  
[008]    t6 = t4 || t5  
[009]    if False t6 goto L3  
[010]    scholarship = 1  
[011]    goto L4
```

```
L3:  
[012]    scholarship = 0
```

```
L4:
```

```
L2:
```

QUADRUPLE TABLE				
#	Op	Arg1	Arg2	Result
0	>	marks	80	t1
1	>	attendance	90	t2
2	&&	t1	t2	t3
3	if False	t3		L1
4	=	2		scholarship
5	goto			L2
6	>	marks	70	t4
7	==	sports	1	t5
8		t4	t5	t6
9	if False	t6		L3
10	=	1		scholarship
11	goto			L4
12	=	0		scholarship

TRIPLE TABLE			
#	Op	Arg1	Arg2
0	>	marks	80
1	>	attendance	90
2	&&	(0)	(1)
3	if False	(2)	L1
4	=	2	scholarship
5	goto	L2	
6	>	marks	70
7	==	sports	1
8		(6)	(7)
9	if False	(8)	L3
10	=	1	scholarship
11	goto	L4	
12	=	0	scholarship

Code 3:


```
PS C:\Users\vishw\Coding\Compiler-Lab\Week-11> .\TAC.exe input3.tx
```

```
Source file : input3.txt
```

```
-----  
i = 0  
found = 0  
while (i < n)  
{  
    if (A[i] == key)  
    {  
        found = 1  
        break  
    }  
    i = i + 1  
}  
-----
```

```
+=====+  
|          THREE-ADDRESS CODE   (Labelled Form)          |  
+=====+
```

```
[000]    i = 0  
[001]    found = 0
```

```
L1:
```

```
[002]    t1 = i < n  
[003]    if False t1 goto L2  
[004]    t2 = A[i]  
[005]    t3 = t2 == key  
[006]    if False t3 goto L3  
[007]    found = 1  
[008]    goto L2
```

```
L3:
```

```
[009]    t4 = i + 1  
[010]    i = t4  
[011]    goto L1
```

```
L2:
```

QUADRUPLE TABLE				
#	Op	Arg1	Arg2	Result
0	=	0		i
1	=	0		found
2	<	i	n	t1
3	if False	t1		L2
4	[]r	A	i	t2
5	==	t2	key	t3
6	if False	t3		L3
7	=	1		found
8	goto			L2
9	+	i	1	t4
10	=	t4		i
11	goto			L1

TRIPLE TABLE			
#	Op	Arg1	Arg2
0	=	0	i
1	=	0	found
2	<	i	n
3	if False	(2)	L2
4	[]r	A	i
5	==	(4)	key
6	if False	(5)	L3
7	=	1	found
8	goto	L2	
9	+	i	1
10	=	(9)	i
11	goto	L1	

Result:

We have successfully implemented a C++ code to find Three Address Code and it's data structures (quadruple, triple).

Compiler Design Lab

Date: 27-3-2026
Assignment No.: 10

Name: S Vishwajith
Register No.: 23BCE1145

Aim:

To reduce the given code using code optimization techniques (Constant folding, Variable propagation, Dead code elimination, code motion, common subexpression elimination, etc.).

Given code:

```
#include <iostream> using
namespace std;

int main() {    int a = 5, b
= 10, c, d, i;    int x = 2 *
3 * 4;    int y = x;    int z =
y;    int unused = a + b;

    if (false) {
        cout << "Hi welcome";

        c = (a * b) + (a * b);
    }    for (i = 0; i < 5; i++)
    {        d = a * b;

int val = i * 2;        cout <<
d + val << endl;

    }
int result = (a * b) + (a * b) + (2 * 3 * 4);

cout << "Final: " << result << endl;
```

```
    return 0;  
}
```

Reduced Code:

```
#include <iostream> int  
  
main() {  
    for (int i = 0; i < 5; i++)  
        std::cout << 50 + (i << 1) << "\n";  
    std::cout << "Final: " << 124 << "\n";  
    return 0;  
}
```

Result:

The given code has been successfully reduced using the techniques, and the reduced code provides the same output as the given code.